

Low-Level Design & CS Fundamentals

A Placement & Interview Companion: OOP, SOLID, Design Patterns,
LLD Case Studies, and Core-Subject Q&A

Maruri Saikiran

August 10, 2026

Contents

1	OOP Foundations and SOLID	1
1.1	Why This Chapter Comes First	1
1.2	The Four Pillars of Object Orientation	2
1.3	Class Relationships and UML Notation	3
1.4	The SOLID Principles	5
2	Design Patterns	9
2.1	How to Choose a Pattern	9
2.2	Singleton	10
2.3	Factory Method	11
2.4	Abstract Factory	12
2.5	Builder	14
2.6	Prototype	15
2.7	Adapter	17
2.8	Bridge	18
2.9	Composite	19
2.10	Decorator	21
2.11	Facade	22
2.12	Flyweight	24
2.13	Proxy	25
2.14	Chain of Responsibility	27
2.15	Command	28
2.16	Iterator	30
2.17	Mediator	31
2.18	Memento	33
2.19	Observer	34
2.20	State	35
2.21	Strategy	37
2.22	Template Method	38
2.23	Visitor	40
3	LLD Case Studies	43
3.1	How to Approach Any LLD Problem	43
3.2	Design a Parking Lot	44
3.3	Design an Elevator System	46
3.4	Design a Vending Machine	47
3.5	Design Tic-Tac-Toe	49
3.6	Design Splitwise (Expense Sharing)	51
3.7	Design a Notification System	53
3.8	Design a Logging Framework	54
3.9	Design a Rate Limiter	56

4	CS Fundamentals: Interview Q&A	59
4.1	How to Use This Chapter	59
4.2	Object-Oriented Programming	60
4.3	Database Management Systems	61
4.4	Operating Systems	63
4.5	Computer Networks	64

Chapter 1

OOP Foundations and SOLID

Low-level design is the craft of turning a vague requirement into a set of classes that are easy to change. Every design pattern and every case study in this book is, at bottom, an application of two things: the four pillars of object orientation, and the five SOLID principles that keep those pillars from cracking as requirements evolve. Learn these first and the rest of the book becomes recognition, not memorisation.

1.1 Why This Chapter Comes First

Interviewers rarely ask “recite the four pillars.” They ask you to *design* something – a parking lot, a notification service, a game – and then watch whether your classes have clear responsibilities, whether new features would force you to rewrite old code, and whether you can justify each choice. Those judgements all trace back to the fundamentals in this chapter.

The One Question Behind All Good Design

Every design decision reduces to: “*when the requirement changes, how much code must I touch, and how likely am I to break something that already works?*” Encapsulation, abstraction, inheritance and polymorphism are the tools; the SOLID principles are the discipline that aims those tools at **minimising the blast radius of change**. Keep this question in mind and you can derive the right answer in an interview instead of recalling one.

How to Recognise Which Principle a Design Smell Violates

Design Smells and the Principle They Break

- **A class that changes for many unrelated reasons**, or a “manager”/“utils” god class – **Single Responsibility**.
- **Adding a new type forces editing a big if/else or switch** on a type field – **Open/Closed** (and usually a missing polymorphic hierarchy or a Factory/Strategy pattern).
- **A subclass that throws “not supported” or weakens a base-class guarantee** (the classic Square-extends-Rectangle) – **Liskov Substitution**.
- **A fat interface whose implementers are forced to stub out methods they do not need** – **Interface Segregation**.
- **High-level policy code that news concrete low-level classes directly**, making it impossible to swap implementations or unit-test – **Dependency Inversion** (inject an abstraction instead).

The three sections of this chapter cover the four pillars, the ways classes relate to one another (association, aggregation, composition, and inheritance, with their UML notation), and the SOLID principles with a concrete before/after refactor for each.

1.2 The Four Pillars of Object Orientation

Problem Statement. Explain encapsulation, abstraction, inheritance, and polymorphism – what each means, why it exists, and how it looks in C++ – since every later pattern is built from them.

What Each Pillar Is Really For

- **Encapsulation** – bundle data with the methods that guard it, and hide the data behind those methods, so invariants cannot be broken from outside.
- **Abstraction** – expose *what* an object does, hide *how*, so callers depend on a stable interface, not internals.
- **Inheritance** – reuse and specialise a base type, modelling an “is-a” relationship.
- **Polymorphism** – let one interface stand for many concrete behaviours chosen at run time, so calling code needn’t know the exact subtype.

Encapsulation and abstraction: two sides of the same wall

Encapsulation is about *protection*: making fields **private** and offering controlled access so no caller can leave an object in an invalid state (a bank balance that only **deposit/withdraw** can change, never a raw assignment). Abstraction is about *simplification*: the caller sees **account.withdraw(100)** and neither knows nor cares how overdraft rules are enforced. Encapsulation hides data; abstraction hides implementation. In practice the same **private** fields plus a small public interface deliver both.

Program to an Interface, Not an Implementation

The single most valuable habit the four pillars enable: callers should depend on an *abstract type* (a base class or interface), so the concrete class behind it can be swapped without touching the caller. This one idea is the seed of polymorphism, the Strategy pattern, dependency injection, and testability.

Inheritance and polymorphism: one call, many behaviours

Inheritance lets a **Dog** reuse and extend an **Animal**. Polymorphism lets a function that accepts an **Animal*** call **speak()** and get barking or meowing depending on the real object, resolved at run time through a **virtual** function. This run-time dispatch is what allows open-ended designs: new animal types slot in without changing the function that iterates over animals.

C++ illustration

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Encapsulation + abstraction: data hidden, invariant enforced.
5 class Account {
6     double balance = 0;                // private: cannot be corrupted
7 public:
8     void deposit(double amt) { if (amt > 0) balance += amt; }
9     bool withdraw(double amt) {
10         if (amt > 0 && amt <= balance) { balance -= amt; return true; }
11         return false;                // overdraft rule hidden inside

```

```

12     }
13     double getBalance() const { return balance; }
14 };
15
16 // Inheritance + polymorphism: one interface, many behaviours.
17 class Animal {
18 public:
19     virtual string speak() const = 0;    // pure virtual: abstraction
20     virtual ~Animal() = default;
21 };
22 class Dog : public Animal {
23 public:
24     string speak() const override { return "Woof"; }
25 };
26 class Cat : public Animal {
27 public:
28     string speak() const override { return "Meow"; }
29 };
30
31 int main() {
32     Account a;
33     a.deposit(100);
34     cout << "Withdrew 40? " << a.withdraw(40) << ", balance "
35          << a.getBalance() << "\n";
36
37     vector<unique_ptr<Animal>> zoo;
38     zoo.push_back(make_unique<Dog>());
39     zoo.push_back(make_unique<Cat>());
40     for (auto& animal : zoo)           // caller knows only "Animal"
41         cout << animal->speak() << " ";
42     cout << "\n";
43     return 0;
44 }

```

Key Takeaway

Encapsulation guards invariants, abstraction hides implementation, inheritance reuses, and polymorphism defers the choice of behaviour to run time. The recurring pay-off is the same: *callers depend on a stable abstraction, so concrete details can change without a ripple*. Every pattern in the next chapter is a specific, named way of exploiting that.

1.3 Class Relationships and UML Notation

Problem Statement. Distinguish the ways two classes can relate – association, aggregation, composition, dependency, and inheritance – and know the UML class-diagram notation, since interviewers expect you to sketch a design and name these relationships correctly.

Which Relationship Is It?

- **“is-a” – inheritance** (a Car is-a Vehicle). UML: hollow triangle arrow to the parent.
- **“has-a,” parts can live independently – aggregation** (a Team has Players; players outlive the team). UML: hollow diamond.
- **“has-a,” parts die with the whole – composition** (a House has Rooms; destroy the house and the rooms go with it). UML: filled diamond.

- “uses-a” transiently – dependency (an Order uses a PaymentGateway passed to one method). UML: dashed arrow.
- A general structural link – association (a plain solid line); aggregation and composition are its stronger, ownership-bearing forms.

The key axis: ownership and lifetime

The distinction interviewers probe is **aggregation vs. composition**, and it is entirely about *lifetime*. In composition the whole *owns* its parts and controls their creation and destruction – model it by storing parts *by value* or via an owning `unique_ptr`. In aggregation the whole merely *references* parts that exist independently – model it by storing a non-owning pointer or reference handed in from outside.

Favour Composition Over Inheritance

A foundational LLD maxim. Inheritance is rigid: it fixes behaviour at compile time and leaks the base class’s details into every subclass. Composing an object out of smaller collaborators (holding a `Strategy`, an `Engine`, a `Logger`) lets you change behaviour by swapping a part at run time. When tempted to subclass, first ask whether “has-a” models the problem better than “is-a.”

C++ illustration of each relationship

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Engine {                      // an independent part
5 public:
6     void start() { cout << "Engine started\n"; }
7 };
8
9 class Logger {                      // dependency: used transiently
10 public:
11     void log(const string& m) { cout << "[log] " << m << "\n"; }
12 };
13
14 // Composition: Car owns its Engine (created and destroyed with the Car).
15 class Car {
16     Engine engine;                  // by value -> composition
17 public:
18     void drive(Logger& logger) {    // dependency: Logger passed in
19         engine.start();
20         logger.log("car driving");
21     }
22 };
23
24 // Aggregation: Team references Players it does not own.
25 class Player {
26 public:
27     string name;
28     Player(string n) : name(move(n)) {}
29 };
30 class Team {

```

```

31     vector<Player*> players;                // non-owning -> aggregation
32 public:
33     void add(Player* p) { players.push_back(p); }
34     void roster() {
35         for (auto* p : players) cout << p->name << " ";
36         cout << "\n";
37     }
38 };
39
40 int main() {
41     Car car; Logger logger;
42     car.drive(logger);
43
44     Player a("Alice"), b("Bob");           // players outlive any team
45     Team t; t.add(&a); t.add(&b);
46     t.roster();
47     return 0;
48 }

```

Key Takeaway

Name relationships precisely: inheritance is “is-a”; composition is owned “has-a” (filled diamond, shared lifetime); aggregation is referenced “has-a” (hollow diamond, independent lifetime); dependency is a transient “uses-a.” In code the tell is ownership – by-value/owning pointer for composition, non-owning pointer for aggregation. When in doubt, prefer composition over inheritance.

1.4 The SOLID Principles

Problem Statement. State each of the five SOLID principles, the design smell it cures, and a concrete before/after in C++. These principles are the vocabulary interviewers use to critique a design, so each is presented as a problem it solves.

SOLID in One Line Each

Single Responsibility – one reason to change per class. **Open/Closed** – open to extension, closed to modification. **Liskov Substitution** – a subtype must be usable anywhere its base is. **Interface Segregation** – many small interfaces beat one fat one. **Dependency Inversion** – depend on abstractions, not concretions.

S – Single Responsibility Principle

A class should have exactly one reason to change. An `Invoice` that computes totals *and* formats itself as PDF *and* emails itself has three reasons to change (tax rules, layout, mail server) and three teams stepping on each other. Split it: `Invoice` (data + totals), `InvoicePrinter` (formatting), `InvoiceMailer` (sending).

O – Open/Closed Principle

Software entities should be open for extension but closed for modification. The smell is a growing `switch` on a type tag:

```

1 // BAD: every new shape edits this function (closed to extension).
2 double area(const Shape& s) {
3     if (s.type == CIRCLE) return 3.14159 * s.r * s.r;

```

```

4     if (s.type == RECTANGLE) return s.w * s.h;
5     // ... adding Triangle means editing here again
6 }

```

Replace the tag with polymorphism so new shapes are *added*, not *edited* in:

```

1 struct Shape { virtual double area() const = 0; virtual ~Shape() =
    default; };
2 struct Circle : Shape {
3     double r; Circle(double r_) : r(r_) {}
4     double area() const override { return 3.14159 * r * r; }
5 };
6 struct Rectangle : Shape {
7     double w, h; Rectangle(double w_, double h_) : w(w_), h(h_) {}
8     double area() const override { return w * h; }
9 };
10 // A new Triangle class needs zero changes to existing code.

```

L – Liskov Substitution Principle

A subclass must honour every promise of its base, so code using the base never breaks when handed the subclass. The classic violation: a **Square** that inherits **Rectangle** and overrides **setWidth** to also change the height silently breaks any code that sets width and height independently and expects them to stay independent. The fix is usually to *not* force the inheritance – model **Square** and **Rectangle** as siblings under a common **Shape**, or use composition.

LSP Is About Behaviour, Not Just Signatures

Compiling is not enough. A subtype substitutes correctly only if it keeps the base's *behavioural contract*: it must not strengthen preconditions, weaken postconditions, or throw “unsupported” for methods the base guarantees. If an override has to disable a base method, the inheritance is wrong.

I – Interface Segregation Principle

No client should be forced to depend on methods it does not use. A fat **Machine** interface with **print()**, **scan()**, **fax()** forces a simple printer to stub out **scan** and **fax**. Split into **Printer**, **Scanner**, **Fax**; a multifunction device implements all three, a basic printer implements only one.

D – Dependency Inversion Principle

High-level modules should depend on abstractions, not on concrete low-level modules – and the abstraction is injected, not constructed inside:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct MessageSender {                                // abstraction
5     virtual void send(const string& msg) = 0;
6     virtual ~MessageSender() = default;
7 };
8 struct EmailSender : MessageSender {
9     void send(const string& msg) override { cout << "Email: " << msg <<
        "\n"; }
10 };
11 struct SmsSender : MessageSender {

```

```
12     void send(const string& msg) override { cout << "SMS: " << msg << "\n"; }
13 };
14
15 class NotificationService {                // high-level policy
16     MessageSender& sender;                  // depends on the abstraction
17 public:
18     NotificationService(MessageSender& s) : sender(s) {} // injected
19     void notify(const string& msg) { sender.send(msg); }
20 };
21
22 int main() {
23     EmailSender email; SmsSender sms;
24     NotificationService viaEmail(email), viaSms(sms);
25     viaEmail.notify("Order shipped");
26     viaSms.notify("OTP 4321");              // swap implementation, no edits
27     return 0;
28 }
```

Key Takeaway

SOLID is one theme in five costumes: *isolate what changes*. SRP isolates responsibilities, OCP isolates new behaviour behind polymorphism, LSP keeps subtypes safe to substitute, ISP isolates clients from irrelevant methods, and DIP isolates high-level policy from low-level detail via an injected abstraction. Every pattern in the next chapter is a reusable structure that upholds one or more of these.

Chapter 2

Design Patterns

A design pattern is a named, reusable answer to a recurring design problem. You will almost never invent one in an interview; you will recognise that the problem in front of you is one you have seen before and reach for the matching pattern. This chapter is organised to make that recognition fast: learn the intent and the tell-tale phrase of each, and the C++ writes itself.

2.1 How to Choose a Pattern

The Gang of Four catalogue splits into three families by *what the pattern is about*: creating objects, composing objects into larger structures, or coordinating behaviour between objects.

The Three Families and Their Job

- **Creational** – control *how objects are made*, so code does not hard-wire concrete classes: Singleton, Factory Method, Abstract Factory, Builder, Prototype.
- **Structural** – compose objects and classes into larger structures while keeping them flexible: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.
- **Behavioural** – assign responsibilities and manage communication between objects: Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor.

Recognise by the Sentence the Interviewer Says

Patterns are triggered by phrases. “Exactly one instance” → Singleton. “Create objects without naming the class” → Factory. “Build a complex object step by step” → Builder. “Add behaviour at run time without subclassing” → Decorator. “Make two incompatible interfaces work together” → Adapter. “Notify many objects when one changes” → Observer. “Swap an algorithm at run time” → Strategy. “Behaviour depends on internal state” → State. Train on the triggers and pattern choice becomes reflexive.

Each section states the pattern’s intent, the phrase that signals it, its participants and structure, a complete C++ example, and the trade-offs that decide whether it is worth the extra indirection.

2.2 Singleton

Problem Statement. Ensure a class has exactly one instance and provide a single global point of access to it – for example a configuration store, a logger, or a connection pool that must be shared, not duplicated.

When You Reach for It

The trigger phrase is *“there must be exactly one”* – one logger, one config, one thread pool. Beware: Singleton is the most overused pattern and effectively a controlled global, so use it only when a second instance would be genuinely wrong, not merely inconvenient.

Understanding the pattern

Three moves enforce singularity: make the constructor **private** so nobody can **new** it, delete the copy constructor and assignment so it cannot be duplicated, and expose a static accessor that lazily creates and returns the one instance. In C++ the cleanest, thread-safe form uses a function-local **static**, whose initialisation the standard guarantees to happen exactly once even under concurrency.

Meyers’ Singleton: a Function-Local Static

A **static** local variable inside the accessor is constructed on first call and destroyed at program exit, and C++11 guarantees this initialisation is thread-safe. This one-liner sidesteps the double-checked locking that plagues Singleton in other languages.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Logger {
5     Logger() { cout << "Logger created\n"; }           // private ctor
6 public:
7     Logger(const Logger&) = delete;                     // no copying
8     Logger& operator=(const Logger&) = delete;
9
10    static Logger& instance() {                          // single access
11        point                                           // created once,
12        static Logger inst;                             thread-safe
13        return inst;
14    }
15    void log(const string& msg) { cout << "[log] " << msg << "\n"; }
16 };
17
18 int main() {
19     Logger::instance().log("first");
20     Logger::instance().log("second");                  // same instance
21     cout << "&a == &b? "
22         << (&Logger::instance() == &Logger::instance()) << "\n";
23     return 0;
24 }
```

Trade-offs

Pros. Guarantees one instance; lazy creation; global access. **Cons.** Introduces global state, which hides dependencies and makes unit testing hard (you cannot easily substitute a mock). It can also become a bottleneck or a hidden coupling point. Prefer dependency injection of a single shared instance when testability matters.

Key Takeaway

Singleton = private constructor + deleted copies + a static accessor. In modern C++ the function-local static gives you thread-safe lazy initialisation for free. Reach for it sparingly; “one instance” is a real constraint, but a shared injected object often serves the same need without the testing pain of a global.

2.3 Factory Method

Problem Statement. Define an interface for creating an object, but let the decision of *which* concrete class to instantiate be made elsewhere, so client code depends only on the abstract product, never on concrete class names.

When You Reach for It

The trigger is “*create an object without hard-coding its class,*” usually to kill a `switch/if-else` on a type string that **news** different concrete classes. Whenever adding a new product type forces you to edit existing creation code, a factory restores the Open/-Closed Principle.

Understanding the pattern

Client code should not contain `new ConcreteA` or `new ConcreteB`. Instead it calls a factory function that takes a request (often an enum or string) and returns a pointer to the abstract product. The concrete decision lives in one place; the rest of the system speaks only the interface. Adding a product means extending the factory, not hunting down every `new` scattered across the codebase.

One Place Knows the Concrete Classes

Centralise object creation behind a function returning the base type. The caller says *what* it wants (“a PDF exporter”); the factory decides *how* to build it. This single indirection is what lets you swap or add implementations without touching callers.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Shape {
5     virtual void draw() const = 0;
6     virtual ~Shape() = default;
7 };
8 struct Circle : Shape { void draw() const override { cout << "Circle\n"
9     ; } };
10 struct Square : Shape { void draw() const override { cout << "Square\n"
11     ; } };

```

```

10
11 enum class ShapeType { Circle, Square };
12
13 struct ShapeFactory {                                // the one place that
14     knows
15     static unique_ptr<Shape> create(ShapeType t) {
16         switch (t) {
17             case ShapeType::Circle: return make_unique<Circle>();
18             case ShapeType::Square: return make_unique<Square>();
19         }
20         return nullptr;
21     };
22
23 int main() {
24     auto s1 = ShapeFactory::create(ShapeType::Circle);
25     auto s2 = ShapeFactory::create(ShapeType::Square);
26     s1->draw();                                     // caller knows only "
27     Shape"
28     s2->draw();
29     return 0;
30 }

```

Trade-offs

Pros. Decouples callers from concrete classes; new products are added in one spot; supports the Open/Closed Principle. **Cons.** Adds a layer of indirection and one more class/function to maintain; the factory itself still centralises knowledge of every product (mitigated by registration-based factories).

Key Takeaway

Factory Method replaces scattered `new ConcreteClass` calls with a single creator returning the abstract type. It is the standard cure for a type-tag `switch` and the most common way an interviewer expects you to satisfy Open/Closed. Abstract Factory (next) generalises it to whole *families* of products.

2.4 Abstract Factory

Problem Statement. Provide an interface for creating *families* of related objects without specifying their concrete classes, so an entire family can be swapped in one move – for example a whole “dark theme” set of UI widgets versus a “light theme” set.

When You Reach for It

The trigger is “*several related objects that must be used together and vary as a set.*” A Windows button must pair with a Windows checkbox, not a Mac one. When products come in matched families and you must guarantee consistency within a family, reach for Abstract Factory.

Understanding the pattern

Where Factory Method creates *one* product, Abstract Factory creates a *suite*. You define an abstract factory interface with a creator method per product type (`createButton`, `createCheckbox`).

Each concrete factory (WindowsFactory, MacFactory) produces the matching family. Client code holds an abstract factory and asks it for products, never knowing or mixing platforms.

A Factory of Factories, One per Family

Group the creators for a coherent family behind a single factory interface. Selecting a concrete factory once (at start-up) fixes the whole family, structurally preventing a mismatch like a Mac button beside a Windows checkbox.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Button { virtual void paint() = 0; virtual ~Button() =
    default; };
5 struct Checkbox { virtual void paint() = 0; virtual ~Checkbox() =
    default; };
6
7 struct WinButton : Button { void paint() override { cout << "Win
    button\n"; } };
8 struct WinCheckbox : Checkbox { void paint() override { cout << "Win
    checkbox\n"; } };
9 struct MacButton : Button { void paint() override { cout << "Mac
    button\n"; } };
10 struct MacCheckbox : Checkbox { void paint() override { cout << "Mac
    checkbox\n"; } };
11
12 struct GUIFactory { // abstract factory
13     virtual unique_ptr<Button> createButton() = 0;
14     virtual unique_ptr<Checkbox> createCheckbox() = 0;
15     virtual ~GUIFactory() = default;
16 };
17 struct WinFactory : GUIFactory {
18     unique_ptr<Button> createButton() override { return make_unique
        <WinButton>(); }
19     unique_ptr<Checkbox> createCheckbox() override { return make_unique
        <WinCheckbox>(); }
20 };
21 struct MacFactory : GUIFactory {
22     unique_ptr<Button> createButton() override { return make_unique
        <MacButton>(); }
23     unique_ptr<Checkbox> createCheckbox() override { return make_unique
        <MacCheckbox>(); }
24 };
25
26 void renderUI(GUIFactory& f) { // platform-agnostic
27     f.createButton()->paint();
28     f.createCheckbox()->paint();
29 }
30
31 int main() {
32     WinFactory win; MacFactory mac;
33     renderUI(win);
34     renderUI(mac); // whole family swapped
35     return 0;
36 }

```

Trade-offs

Pros. Guarantees products from one family are used together; swapping families is a one-line change; isolates concrete classes. **Cons.** Adding a *new product type* means changing the factory interface and every concrete factory – a real cost when the product set grows. Best when families change often but the set of product types is stable.

Key Takeaway

Abstract Factory is Factory Method scaled to families: one factory interface with several creators, one concrete factory per family. It buys family consistency at the price of rigidity when new product *types* appear. Recognise it whenever “these objects must all match” is a requirement.

2.5 Builder

Problem Statement. Separate the construction of a complex object from its representation, so the same step-by-step process can build different configurations – especially useful when an object has many optional parameters.

When You Reach for It

The triggers are “*an object with many optional fields*” (the telescoping-constructor smell: `Pizza(true,false,true,...)`) and “*build a complex object in steps.*” When a constructor has a long list of parameters, half of them optional, a Builder makes construction readable and safe.

Understanding the pattern

Instead of one giant constructor, a Builder exposes chainable setter methods (`addCheese()`, `addOlives()`) that accumulate configuration, then a `build()` that produces the finished, immutable product. Each setter returns the builder itself so calls read like a sentence. The product’s own constructor can stay private, taking only the builder, guaranteeing objects are never half-initialised.

Chainable Steps, One Final Build

Name each optional step explicitly and let the terminal `build()` assemble the product. This turns an unreadable positional-argument call into self-documenting, order-independent construction, and lets you validate the whole configuration in one place before creating the object.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Pizza {
5 public:
6     string size;
7     bool cheese = false, olives = false, mushrooms = false;
8     void describe() const {
9         cout << size << " pizza"
10             << (cheese ? " +cheese" : "")

```

```

11         << (olives ? " +olives" : "")
12         << (mushrooms ? " +mushrooms" : "") << "\n";
13     }
14 };
15
16 class PizzaBuilder {
17     Pizza p;
18 public:
19     PizzaBuilder& setSize(const string& s) { p.size = s; return *this;
20     }
21     PizzaBuilder& addCheese()      { p.cheese = true;      return *this; }
22     PizzaBuilder& addOlives()     { p.olives = true;      return *this; }
23     PizzaBuilder& addMushrooms() { p.mushrooms = true; return *this; }
24     Pizza build() { return p; } // finished product
25 };
26
27 int main() {
28     Pizza pizza = PizzaBuilder()
29                 .setSize("Large")
30                 .addCheese()
31                 .addMushrooms()
32                 .build(); // reads like a sentence
33     pizza.describe();
34     return 0;
35 }

```

Trade-offs

Pros. Readable, order-independent construction; no telescoping constructors; can enforce validation and immutability at build time. **Cons.** More code than a plain constructor; overkill for objects with a couple of fields. Worth it precisely when the parameter list is long and largely optional.

Key Takeaway

Builder trades a long, error-prone constructor for a fluent chain of named steps ending in `build()`. Recognise it by the smell of many optional parameters or boolean flags at a call site. It is one of the most frequently *used* patterns in real code, not just interviews.

2.6 Prototype

Problem Statement. Create new objects by cloning an existing prototype rather than constructing from scratch, useful when object creation is expensive or when the exact concrete type to copy is known only at run time.

When You Reach for It

The trigger is “*make a copy of an existing configured object,*” or “*creating one from scratch is costly.*” When you have a fully-configured instance and want more just like it – without knowing or naming its concrete class – a polymorphic `clone()` is the answer.

Understanding the pattern

Each class implements a virtual `clone()` that returns a copy of itself as the base type. Client code holds a base pointer and calls `clone()`; it gets back a new object of the correct concrete

type without a `switch` on types or a call to a specific constructor. This is invaluable when the set of concrete classes is registered at run time (a palette of prototype shapes the user can duplicate).

Let the Object Copy Itself

Because only the object knows its own concrete type, delegate copying to a virtual `clone()`. This sidesteps “which constructor do I call?” and preserves all the run-time state of the prototype, including fields a factory would not know to set.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Shape {
5     int x = 0, y = 0;
6     virtual unique_ptr<Shape> clone() const = 0;    // polymorphic copy
7     virtual void draw() const = 0;
8     virtual ~Shape() = default;
9 };
10
11 struct Circle : Shape {
12     int r = 1;
13     unique_ptr<Shape> clone() const override {
14         return make_unique<Circle>(*this);        // copy all state
15     }
16     void draw() const override {
17         cout << "Circle r=" << r << " at (" << x << ", " << y << ")\n";
18     }
19 };
20
21 int main() {
22     Circle base; base.r = 5; base.x = 2; base.y = 3;
23
24     unique_ptr<Shape> proto = make_unique<Circle>(base);
25     auto copy1 = proto->clone();                  // no concrete type
26     // named
27     auto copy2 = proto->clone();
28     copy1->draw();
29     copy2->draw();
30     return 0;
31 }

```

Trade-offs

Pros. Copies run-time state exactly; avoids costly re-construction; adds objects without naming concrete classes. **Cons.** Deep vs. shallow copy must be handled carefully (pointers, cycles); every class must implement `clone()` correctly. Watch out for objects owning raw pointers.

Key Takeaway

Prototype creates by copying a live object through a virtual `clone()`, sidestepping constructors and type switches. Reach for it when you already have a configured instance and want duplicates, or when construction is expensive. The one hazard is getting deep-copy

semantics right.

2.7 Adapter

Problem Statement. Convert the interface of an existing class into another interface that clients expect, letting classes that could not otherwise collaborate – because of mismatched method names or signatures – work together.

When You Reach for It

The trigger is “*I have a class that does the right thing but has the wrong interface,*” often a third-party or legacy library you cannot change. When the client expects `request()` but the library offers `specificRequest()`, an Adapter bridges the gap.

Understanding the pattern

The Adapter implements the interface the client wants and, internally, holds an instance of the incompatible class (object adapter, via composition). Each method translates the client’s call into one or more calls on the wrapped object. The client codes against the target interface; the adapter absorbs the mismatch.

Wrap the Wrong Interface in the Right One

Compose the incompatible “adaptee” inside a class that presents the expected interface, forwarding and translating calls. Nothing about the adaptee or the client changes – the adapter is the only new, and only throwaway-if-needed, piece.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // Target: what the client expects.
5  struct MediaPlayer {
6      virtual void play(const string& file) = 0;
7      virtual ~MediaPlayer() = default;
8  };
9
10 // Adaptee: useful, but wrong interface (cannot be modified).
11 struct LegacyMp4Engine {
12     void playMp4(const string& filename) {
13         cout << "Playing MP4: " << filename << "\n";
14     }
15 };
16
17 // Adapter: presents MediaPlayer, delegates to the legacy engine.
18 class Mp4Adapter : public MediaPlayer {
19     LegacyMp4Engine engine;           // composed adaptee
20 public:
21     void play(const string& file) override {
22         engine.playMp4(file);         // translate the call
23     }
24 };
25

```

```

26 int main() {
27     unique_ptr<MediaPlayer> player = make_unique<Mp4Adapter>();
28     player->play("song.mp4");           // client uses target
29     API
30     return 0;
31 }

```

Trade-offs

Pros. Reuses existing/legacy code without modifying it; isolates interface-conversion logic in one place; supports Single Responsibility. **Cons.** Adds an extra layer; if many methods need adapting the adapter grows. A sign the underlying interfaces should perhaps be redesigned if you control both.

Key Takeaway

Adapter makes an existing class fit an expected interface by wrapping and translating – “right behaviour, wrong signature.” Contrast with Decorator (same interface, added behaviour) and Facade (new simpler interface over many classes): all three wrap, but for different reasons.

2.8 Bridge

Problem Statement. Decouple an abstraction from its implementation so the two can vary independently, avoiding a combinatorial explosion of subclasses when a type varies along two separate dimensions.

When You Reach for It

The trigger is “*two independent dimensions of variation*” that, combined by inheritance, would create $m \times n$ subclasses. Shapes (circle, square) times rendering APIs (OpenGL, DirectX) would need `OpenGLCircle`, `DirectXCircle`, `OpenGLSquare`... Bridge turns that product into a sum.

Understanding the pattern

Split the two dimensions into two class hierarchies and connect them with a reference (the “bridge”). The *abstraction* hierarchy (shapes) holds a pointer to an *implementor* hierarchy (renderers) and delegates the primitive operations. Now you add shapes and renderers independently: $m + n$ classes instead of $m \times n$.

Prefer Composition Across the Two Axes

When a class varies along two axes, do not multiply them with inheritance. Give one axis (the abstraction) a member pointing at the other (the implementation). This is “favour composition over inheritance” applied to prevent a subclass explosion.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Implementor axis: how to render.
5 struct Renderer {

```

```

6     virtual void renderCircle(float r) = 0;
7     virtual ~Renderer() = default;
8 };
9 struct OpenGLRenderer : Renderer {
10     void renderCircle(float r) override { cout << "OpenGL circle r=" <<
        r << "\n"; }
11 };
12 struct DirectXRenderer : Renderer {
13     void renderCircle(float r) override { cout << "DirectX circle r="
        << r << "\n"; }
14 };
15
16 // Abstraction axis: what to draw. Holds a bridge to the implementor.
17 class Shape {
18 protected:
19     Renderer& renderer;                // the bridge
20 public:
21     Shape(Renderer& r) : renderer(r) {}
22     virtual void draw() = 0;
23     virtual ~Shape() = default;
24 };
25 class Circle : public Shape {
26     float radius;
27 public:
28     Circle(Renderer& r, float rad) : Shape(r), radius(rad) {}
29     void draw() override { renderer.renderCircle(radius); }
30 };
31
32 int main() {
33     OpenGLRenderer gl; DirectXRenderer dx;
34     Circle c1(gl, 5), c2(dx, 8);        // mix axes freely
35     c1.draw();
36     c2.draw();
37     return 0;
38 }

```

Trade-offs

Pros. Abstraction and implementation evolve independently; avoids class explosion; swappable implementations at run time. **Cons.** More indirection and up-front structure; only pays off when *both* dimensions genuinely vary. For a single axis it is over-engineering.

Key Takeaway

Bridge separates “what” from “how” into two hierarchies joined by a reference, turning an $m \times n$ subclass explosion into $m + n$. The signal is two orthogonal dimensions of change. It looks structurally like Strategy but its intent is structural decoupling, not swapping an algorithm.

2.9 Composite

Problem Statement. Compose objects into tree structures to represent part-whole hierarchies, and let clients treat individual objects and compositions of objects uniformly through the same interface.

When You Reach for It

The trigger is “a tree of objects where leaves and containers should be handled the same way” – files and folders, a GUI of widgets containing widgets, an org chart. When client code should not care whether it holds one item or a group, use Composite.

Understanding the pattern

Define one *component* interface implemented by both *leaves* (no children) and *composites* (hold children, also components). A composite’s operation typically recurses over its children. Because both share the interface, client code calls, say, `getSize()` on anything and the recursion through the tree happens transparently.

Leaf and Container Share One Interface

Make the container a component too, holding a list of components. An operation on a container delegates to its children and aggregates. This uniformity is the whole point: no `if (isFolder)` branching in client code.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct FileSystemNode {                                // component
5      virtual int size() const = 0;
6      virtual void print(string indent = "") const = 0;
7      virtual ~FileSystemNode() = default;
8  };
9
10 struct File : FileSystemNode {                          // leaf
11     string name; int bytes;
12     File(string n, int b) : name(move(n)), bytes(b) {}
13     int size() const override { return bytes; }
14     void print(string indent) const override {
15         cout << indent << name << " (" << bytes << ")\n";
16     }
17 };
18
19 struct Folder : FileSystemNode {                        // composite
20     string name;
21     vector<unique_ptr<FileSystemNode>> children;
22     Folder(string n) : name(move(n)) {}
23     void add(unique_ptr<FileSystemNode> c) { children.push_back(move(c)); }
24     int size() const override {
25         int total = 0;
26         for (auto& c : children) total += c->size(); // recurse
27         return total;
28     }
29     void print(string indent) const override {
30         cout << indent << name << "\n";
31         for (auto& c : children) c->print(indent + " ");
32     }
33 };
34

```

```

35 int main() {
36     auto root = make_unique<Folder>("root");
37     root->add(make_unique<File>("a.txt", 100));
38     auto sub = make_unique<Folder>("sub");
39     sub->add(make_unique<File>("b.txt", 250));
40     root->add(move(sub));
41
42     root->print();
43     cout << "Total size: " << root->size() << "\n";    // treated
44     uniformly
45     return 0;
46 }

```

Trade-offs

Pros. Uniform treatment of leaves and groups; new node types slot in easily; recursion over the tree is natural. **Cons.** The shared interface can become overly general (leaf-only or container-only methods awkwardly appear on both); type safety of “only folders have children” is relaxed.

Key Takeaway

Composite models part-whole trees behind a single component interface so clients treat one and many identically, with container operations recursing over children. Recognise it wherever a hierarchy of “things and groups of things” must be processed uniformly.

2.10 Decorator

Problem Statement. Attach additional responsibilities to an object dynamically, at run time, without subclassing – providing a flexible alternative to inheritance for extending behaviour.

When You Reach for It

The trigger is “*add behaviour at run time in combinations*,” where a subclass per combination would explode (coffee with milk, with sugar, with milk-and-sugar-and-cream). When features stack in arbitrary combinations, wrap rather than subclass.

Understanding the pattern

A decorator implements the *same interface* as the object it wraps and holds a pointer to a wrapped component of that interface. It forwards calls to the inner object and adds its own behaviour before or after. Since a decorator is itself a component, decorators nest: each layer adds one feature, and you compose features by stacking wrappers at run time.

Same Interface, Wrap and Add

Unlike Adapter (changes the interface) a Decorator keeps the interface identical and augments the behaviour, so wrapped and unwrapped objects are interchangeable. Stacking decorators composes features additively without a single new subclass.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

3
4 struct Coffee {                                     // component
5     virtual double cost() const = 0;
6     virtual string desc() const = 0;
7     virtual ~Coffee() = default;
8 };
9
10 struct Espresso : Coffee {                         // concrete component
11     double cost() const override { return 2.0; }
12     string desc() const override { return "Espresso"; }
13 };
14
15 struct CoffeeDecorator : Coffee {                  // base decorator
16     unique_ptr<Coffee> inner;
17     CoffeeDecorator(unique_ptr<Coffee> c) : inner(move(c)) {}
18 };
19
20 struct Milk : CoffeeDecorator {
21     Milk(unique_ptr<Coffee> c) : CoffeeDecorator(move(c)) {}
22     double cost() const override { return inner->cost() + 0.5; }
23     string desc() const override { return inner->desc() + " + Milk"; }
24 };
25 struct Sugar : CoffeeDecorator {
26     Sugar(unique_ptr<Coffee> c) : CoffeeDecorator(move(c)) {}
27     double cost() const override { return inner->cost() + 0.2; }
28     string desc() const override { return inner->desc() + " + Sugar"; }
29 };
30
31 int main() {
32     unique_ptr<Coffee> order =
33         make_unique<Sugar>(make_unique<Milk>(make_unique<Espresso>()));
34     cout << order->desc() << " = $" << order->cost() << "\n"; //
35         stacked
36     return 0;
37 }

```

Trade-offs

Pros. Add/remove responsibilities at run time; avoids a subclass per feature-combination; each decorator is a small, single-purpose class. **Cons.** Many small objects and layers can be hard to debug; order of wrapping can matter; a deep stack obscures the concrete object.

Key Takeaway

Decorator wraps an object in another of the same interface to add behaviour, and nests to compose features – the run-time alternative to a combinatorial subclass tree. Keyword: “add responsibilities dynamically.” It is the pattern behind Java’s I/O streams and many middleware chains.

2.11 Facade

Problem Statement. Provide a single, simplified interface to a complex subsystem of many classes, so common tasks become easy while the full subsystem remains available for advanced use.

When You Reach for It

The trigger is “*clients must call five classes in the right order just to do one common thing.*” When using a subsystem requires knowing too much of its internals, a Facade offers a clean front door for the typical case.

Understanding the pattern

The Facade is a class that knows the subsystem’s classes and orchestrates them. It exposes a few high-level methods (`watchMovie()`) that internally coordinate the amplifier, projector, lights and player. Clients talk to the Facade; they are shielded from the subsystem’s structure and call order. Crucially the subsystem is not hidden – power users can still reach the underlying classes directly.

A Simple Front Door, Not a Wall

A Facade reduces coupling between clients and a subsystem by centralising the common orchestration in one place. It does not add behaviour or restrict access; it simply makes the easy things easy while leaving the hard things possible.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Amplifier { void on() { cout << "Amp on\n"; } void setVolume(int
   v) { cout << "Volume " << v << "\n"; } };
5 struct Projector { void on() { cout << "Projector on\n"; } void wide()
   { cout << "Widescreen\n"; } };
6 struct Lights    { void dim(int p) { cout << "Lights " << p << "%\n"; }
   };
7 struct Player    { void play(const string& m) { cout << "Playing " << m
   << "\n"; } };
8
9 class HomeTheaterFacade {                                // the simple front
   door
10     Amplifier amp; Projector projector; Lights lights; Player player;
11 public:
12     void watchMovie(const string& movie) {                // one call
   orchestrates all
13         lights.dim(10);
14         projector.on();
15         projector.wide();
16         amp.on();
17         amp.setVolume(7);
18         player.play(movie);
19     }
20 };
21
22 int main() {
23     HomeTheaterFacade theater;
24     theater.watchMovie("Inception");                       // client does one
   thing
25     return 0;
26 }

```

Trade-offs

Pros. Simplifies client code; decouples clients from subsystem internals; a natural place to enforce a correct call sequence. **Cons.** Can become a “god object” if it accumulates too much; does not prevent misuse of the subsystem directly. Keep it thin – a coordinator, not a kitchen sink.

Key Takeaway

Facade wraps a complicated subsystem behind a small, task-oriented interface, lowering the knowledge a client needs. Distinguish from Adapter (converts one interface to another expected one) – Facade invents a *new, simpler* interface over *many* classes.

2.12 Flyweight

Problem Statement. Minimise memory use by sharing as much data as possible between many similar objects, separating the intrinsic (shared, state-independent) part from the extrinsic (per-instance) part.

When You Reach for It

The trigger is “*millions of near-identical objects blowing up memory*” – every character in a document, every tree in a forest, every bullet in a game. When objects share most of their data, store the shared part once and pass in what differs.

Understanding the pattern

Partition an object’s state into **intrinsic** state – shared, immutable, independent of context (a glyph’s shape and font) – and **extrinsic** state – unique per use (the glyph’s position on the page). Store one flyweight per distinct intrinsic value in a factory/pool, hand the same shared flyweight to every user, and pass extrinsic state as method arguments rather than storing it.

Share the Intrinsic, Pass the Extrinsic

Cache and reuse the heavy shared part keyed by its value; supply the light per-instance part at call time. A thousand ‘a’ characters point to one shared glyph object; only their coordinates differ and those live in the document, not in a thousand glyph copies.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Glyph {                                // flyweight: intrinsic
5     state                                     // shared, immutable
6     char symbol;
7 public:
8     Glyph(char c) : symbol(c) { cout << "Created glyph '" << c << "'\n"
9         ; }
10    void draw(int x, int y) const {           // extrinsic passed in
11        cout << "'" << symbol << "' at (" << x << ", " << y << ")\n";
12    }
13 };

```

```

13 class GlyphFactory {                                     // ensures sharing
14     unordered_map<char, shared_ptr<Glyph>> pool;
15 public:
16     shared_ptr<Glyph> get(char c) {
17         auto it = pool.find(c);
18         if (it != pool.end()) return it->second;         // reuse
19         auto g = make_shared<Glyph>(c);
20         pool[c] = g;
21         return g;
22     }
23 };
24
25 int main() {
26     GlyphFactory factory;
27     string text = "aab";
28     int x = 0;
29     for (char c : text) {
30         auto g = factory.get(c);                         // 'a' created once,
31         shared                                           shared
32         g->draw(x, 0);
33         x += 10;
34     }
35     return 0;

```

Trade-offs

Pros. Dramatic memory savings when many objects share state; fewer allocations. **Cons.** Complexity of splitting state; extrinsic state must be recomputed or passed each time (a CPU-for-memory trade); flyweights must be immutable to be safely shared.

Key Takeaway

Flyweight shares one immutable intrinsic object among many contexts and passes the varying extrinsic state as arguments, trading a little CPU for large memory savings. Recognise it by “huge number of objects, mostly identical.” The pool/factory that guarantees sharing is essential.

2.13 Proxy

Problem Statement. Provide a surrogate or placeholder for another object to control access to it – for lazy initialisation, access control, caching, logging, or standing in for a remote object.

When You Reach for It

The trigger is “wrap an object to control how and when it is used” without changing its interface: delay an expensive load until needed (virtual proxy), check permissions (protection proxy), cache results, or represent something across a network (remote proxy).

Understanding the pattern

A Proxy implements the *same interface* as the real subject and holds a reference to it. It intercepts each call, does its extra job (lazy create, permission check, cache lookup, log), then forwards to the real object. Because the interface matches, clients cannot tell they are talking to a proxy rather than the real thing.

Same Interface, Controlled Access

Proxy shares Decorator's "wrap with the same interface" shape but its intent is *access control and management*, not adding features. The canonical use is the virtual proxy: construct the heavy real object only on first genuine use.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Image {
5     virtual void display() = 0;
6     virtual ~Image() = default;
7 };
8
9 class RealImage : public Image {                // expensive real
10     subject
11     string filename;
12 public:
13     RealImage(const string& f) : filename(f) {
14         cout << "Loading " << filename << " from disk\n";    // heavy
15     }
16     void display() override { cout << "Displaying " << filename << "\n"
17         ; }
18 };
19
20 class ImageProxy : public Image {                // virtual proxy
21     string filename;
22     unique_ptr<RealImage> real;                  // created lazily
23 public:
24     ImageProxy(const string& f) : filename(f) {}
25     void display() override {
26         if (!real) real = make_unique<RealImage>(filename);    // load on
27         demand
28         real->display();
29     }
30 };
31
32 int main() {
33     ImageProxy img("photo.png");                // no disk load yet
34     cout << "Proxy created, nothing loaded.\n";
35     img.display();                               // loads now, then
36         displays
37     img.display();                               // reuses, no reload
38     return 0;
39 }

```

Trade-offs

Pros. Adds access control, laziness, caching or remotng transparently; the client interface is unchanged. **Cons.** Extra indirection and latency on first access; a poorly designed proxy can hide performance costs or introduce inconsistency between proxy and subject.

Key Takeaway

Proxy stands in for a real object behind an identical interface to control access – lazy loading, protection, caching, or remoting. It looks like Decorator but the intent differs: Decorator *adds behaviour*, Proxy *governs access*. Name the proxy variant (virtual, protection, remote) when you use it.

2.14 Chain of Responsibility

Problem Statement. Pass a request along a chain of handlers; each handler decides either to process the request or to forward it to the next, decoupling the sender from the specific receiver that ends up handling it.

When You Reach for It

The trigger is “a request that several handlers might process, and we don’t know in advance which one will.” Approval workflows, event bubbling, logging levels, and middleware pipelines all fit: try each handler in turn until one takes it.

Understanding the pattern

Each handler holds a pointer to the next and implements a `handle(request)`: if it can deal with the request it does; otherwise it delegates to the next in line. The client fires the request at the head of the chain and never needs to know which link resolves it. Handlers can be reordered or inserted without touching the sender.

Handle or Pass Along

Give every handler the same interface and a “next” link. The decision “mine or not mine” is local to each handler; the chain’s shape is configured externally. This decouples *who sends* from *who handles*.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Approver {
5 protected:
6     Approver* next = nullptr;
7 public:
8     Approver* setNext(Approver* n) { next = n; return n; }
9     virtual void handle(int amount) {
10         if (next) next->handle(amount);           // default: pass along
11         else cout << "No one could approve " << amount << "\n";
12     }
13     virtual ~Approver() = default;
14 };
15
16 class Manager : public Approver {
17 public:
18     void handle(int amount) override {
19         if (amount <= 1000) cout << "Manager approves " << amount << "\n";
20     }
21 }

```

```

20         else Approver::handle(amount);
21     }
22 };
23 class Director : public Approver {
24 public:
25     void handle(int amount) override {
26         if (amount <= 10000) cout << "Director approves " << amount <<
27             "\n";
28         else Approver::handle(amount);
29     }
30 };
31 int main() {
32     Manager mgr; Director dir;
33     mgr.setNext(&dir);                // build the chain
34     mgr.handle(500);                  // Manager
35     mgr.handle(5000);                 // passed to Director
36     mgr.handle(50000);                // unhandled
37     return 0;
38 }

```

Trade-offs

Pros. Decouples sender and receiver; handlers are added, removed, or reordered freely; each handler is simple. **Cons.** A request may fall off the end unhandled; harder to trace which handler acted; a long chain adds latency. Ensure a terminal/default handler.

Key Takeaway

Chain of Responsibility lets a request travel handler-to-handler until one processes it, decoupling the caller from the resolver. Recognise it in approval hierarchies and middleware. The key risk to mention: unhandled requests, so design an end-of-chain default.

2.15 Command

Problem Statement. Encapsulate a request as an object, so requests can be parameterised, queued, logged, and undone – decoupling the object that invokes an operation from the one that knows how to perform it.

When You Reach for It

The triggers are “undo/redo,” “queue or schedule operations,” and “parameterise a button/menu with an action.” When an action must be treated as a first-class value – stored, passed, reversed – wrap it in a Command object.

Understanding the pattern

A Command object exposes `execute()` (and often `undo()`) and holds everything the action needs: a reference to the receiver and any arguments. An invoker (a button, a queue, a macro) triggers commands without knowing what they do. Because each command is an object, you can keep a history stack for undo, replay a log, or batch commands into a macro.

Turn a Method Call into an Object

Reifying “call this method with these arguments” as an object is what enables undo, queuing, logging and macros – none of which are possible with a bare function call. The invoker depends only on the Command interface.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Light {                                     // receiver
5     bool on = false;
6     void turnOn() { on = true; cout << "Light on\n"; }
7     void turnOff() { on = false; cout << "Light off\n"; }
8 };
9
10 struct Command {
11     virtual void execute() = 0;
12     virtual void undo() = 0;
13     virtual ~Command() = default;
14 };
15
16 struct LightOnCommand : Command {
17     Light& light;
18     LightOnCommand(Light& l) : light(l) {}
19     void execute() override { light.turnOn(); }
20     void undo() override { light.turnOff(); }
21 };
22
23 class RemoteControl {                             // invoker with undo
24     vector<unique_ptr<Command>> history;
25 public:
26     void submit(unique_ptr<Command> cmd) {
27         cmd->execute();
28         history.push_back(move(cmd));
29     }
30     void undoLast() {
31         if (!history.empty()) {
32             history.back()->undo();
33             history.pop_back();
34         }
35     }
36 };
37
38 int main() {
39     Light light;
40     RemoteControl remote;
41     remote.submit(make_unique<LightOnCommand>(light));
42     remote.undoLast();                             // reverses it
43     return 0;
44 }

```

Trade-offs

Pros. Enables undo/redo, queuing, logging, and macros; decouples invoker from receiver; commands are composable. **Cons.** A class per action can proliferate; undo requires each command to capture enough state to reverse itself, which can be non-trivial.

Key Takeaway

Command packages an action plus its receiver and arguments into an object with `execute/undo`, unlocking undo stacks, queues, and macros. The interview keyword is almost always “undo/redo.” It is the backbone of editors and task schedulers.

2.16 Iterator

Problem Statement. Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation, so clients traverse a collection the same way regardless of how it is stored.

When You Reach for It

The trigger is “*traverse a collection without revealing whether it’s an array, list, or tree.*” When you want uniform iteration and the freedom to change the container’s internals, expose an iterator instead of raw structure.

Understanding the pattern

Define an iterator interface with `hasNext()` and `next()`. The collection produces an iterator that knows how to walk *its* particular structure. Client loops depend only on the iterator interface, so swapping a vector for a linked list – or offering multiple traversal orders – does not touch client code. This is the pattern the C++ STL and range-based `for` are built on.

Externalise Traversal State

Move the “where am I in the collection” cursor out of the client and into a dedicated iterator object. Multiple independent iterators can then traverse the same collection at once, and the collection’s storage stays hidden.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template <typename T>
5 struct Iterator {
6     virtual bool hasNext() const = 0;
7     virtual T next() = 0;
8     virtual ~Iterator() = default;
9 };
10
11 class IntCollection {                                // hides its storage
12     vector<int> data;
13 public:
14     void add(int x) { data.push_back(x); }
15 
```

```

16     class Iter : public Iterator<int> {
17         const vector<int>& ref; size_t pos = 0;
18     public:
19         Iter(const vector<int>& r) : ref(r) {}
20         bool hasNext() const override { return pos < ref.size(); }
21         int next() override { return ref[pos++]; }
22     };
23
24     unique_ptr<Iterator<int>> iterator() const {
25         return make_unique<Iter>(data);
26     }
27 };
28
29 int main() {
30     IntCollection c;
31     c.add(10); c.add(20); c.add(30);
32     auto it = c.iterator();
33     while (it->hasNext()) cout << it->next() << " ";    // storage
34     cout << "\n";
35     return 0;
36 }

```

Trade-offs

Pros. Uniform traversal; hides representation; supports multiple simultaneous and multiple-order traversals; simplifies the collection's own interface. **Cons.** Extra objects for simple cases; an iterator can be invalidated if the collection is modified during traversal.

Key Takeaway

Iterator externalises traversal into an object with `hasNext/next`, decoupling clients from a collection's internal structure. It is so fundamental that most languages bake it in (STL iterators, `for-each`). Mention iterator invalidation as the classic pitfall.

2.17 Mediator

Problem Statement. Define an object that encapsulates how a set of objects interact, so they refer to the mediator instead of to each other – turning a tangle of many-to-many references into a hub-and-spoke.

When You Reach for It

The trigger is “*every component talks to every other component*” – an n^2 web of direct references (a dialog where each widget reacts to others, a chat room, air-traffic control). When coupling between peers grows quadratic, route it through a mediator.

Understanding the pattern

Instead of colleagues holding references to one another, each holds a reference to a single *mediator*. When a colleague changes, it notifies the mediator, which decides who else needs to know and calls them. The colleagues become simple and reusable; all the interaction logic concentrates in one place – the mediator – which is the only object that knows the participants.

Centralise the Many-to-Many

Replace peer-to-peer links with a hub. Colleagues know only the mediator; the mediator knows the colleagues. This converts $O(n^2)$ coupling into $O(n)$ and makes interaction rules explicit and changeable in one spot.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class User;
5 struct ChatRoom {                                // mediator
6     virtual void send(const string& msg, User* from) = 0;
7     virtual ~ChatRoom() = default;
8 };
9
10 class User {
11     string name;
12     ChatRoom* room;                               // knows only the
13     mediator
14 public:
15     User(string n, ChatRoom* r) : name(move(n)), room(r) {}
16     const string& getName() const { return name; }
17     void send(const string& msg);
18     void receive(const string& msg, const string& from) {
19         cout << name << " got from " << from << ": " << msg << "\n";
20     }
21 };
22
23 class Room : public ChatRoom {
24     vector<User*> users;
25 public:
26     void join(User* u) { users.push_back(u); }
27     void send(const string& msg, User* from) override {
28         for (User* u : users)
29             if (u != from) u->receive(msg, from->getName()); // hub
30         routes
31     }
32 };
33
34 void User::send(const string& msg) { room->send(msg, this); }
35
36 int main() {
37     Room room;
38     User a("Alice", &room), b("Bob", &room), c("Carol", &room);
39     room.join(&a); room.join(&b); room.join(&c);
40     a.send("hi all"); // no direct peer links
41     return 0;
42 }

```

Trade-offs

Pros. Removes tight peer-to-peer coupling; interaction logic lives in one place; colleagues become reusable. **Cons.** The mediator can swell into a complex “god object” as rules accumulate – you have centralised complexity, not removed it.

Key Takeaway

Mediator replaces a many-to-many mesh of collaborators with a hub that owns the interaction rules, cutting coupling from $O(n^2)$ to $O(n)$. Watch the mediator itself for bloat. Classic examples: chat rooms, UI dialog coordination, air-traffic control.

2.18 Memento

Problem Statement. Capture and externalise an object’s internal state without violating encapsulation, so the object can be restored to that state later – the mechanism behind save points and undo.

When You Reach for It

The trigger is “*save and restore an object’s state*” – snapshots, checkpoints, undo history. When you must be able to roll an object back but do not want to expose its internals to the code that stores the snapshot, use a Memento.

Understanding the pattern

Three roles: the **Originator** (whose state we snapshot) creates a **Memento** holding a copy of its state and can later restore from one; the **Caretaker** keeps mementos (e.g. on an undo stack) but treats them as opaque – it never reads their contents. Encapsulation is preserved because only the Originator can read or write a memento’s internals.

Opaque Snapshots the Caretaker Cannot Read

The Originator packages its own state into a memento and unpacks it on restore; the Caretaker only stores and hands mementos back. This division gives you undo without leaking private state to the code that manages history.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Memento {                                // opaque snapshot
5      friend class Editor;                       // only Editor sees
6      string state;
7      Memento(string s) : state(move(s)) {}
8  };
9
10 class Editor {                                  // originator
11     string content;
12 public:
13     void type(const string& text) { content += text; }
14     string show() const { return content; }
15     Memento save() const { return Memento(content); } // snapshot
16     void restore(const Memento& m) { content = m.state; } // roll back
17 };
18
19 int main() {
20     Editor editor;

```

```

21 editor.type("Hello");
22 Memento checkpoint = editor.save();           // caretaker holds this
23 editor.type(", World");
24 cout << "Before undo: " << editor.show() << "\n";
25 editor.restore(checkpoint);                   // back to "Hello"
26 cout << "After undo: " << editor.show() << "\n";
27 return 0;
28 }

```

Trade-offs

Pros. Clean save/restore without breaking encapsulation; pairs naturally with Command for undo. **Cons.** Snapshots can be memory-heavy if state is large or saved often; deciding what to include in a memento and how frequently to snapshot needs care (consider incremental/diff mementos).

Key Takeaway

Memento snapshots an object's state into an opaque object that only the originator can interpret, enabling undo and checkpoints while keeping internals private. It frequently teams with Command: the command stores a memento to reverse itself. Cost to flag: memory for many/large snapshots.

2.19 Observer

Problem Statement. Define a one-to-many dependency so that when one object (the subject) changes state, all its dependents (observers) are notified and updated automatically – the foundation of event systems and publish/subscribe.

When You Reach for It

The trigger is *“when X changes, several other things must react”* – a spreadsheet cell and its charts, a stock price and its displays, a model and its views. When one source has many interested listeners that come and go, use Observer.

Understanding the pattern

The **Subject** maintains a list of **Observers** and offers `subscribe/unsubscribe`. On a relevant change it calls `notify()`, which invokes `update()` on every observer. The subject knows only the observer *interface*, so observers can be added or removed at run time without the subject knowing their concrete types. This is the classic Model-View decoupling.

Publish to a List of Subscribers

Keep subscribers behind an interface and iterate them on change. Loose coupling is the point: the subject broadcasts an event; who listens, and how many, is not its concern and can vary dynamically.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Observer {

```

```

5     virtual void update(int price) = 0;
6     virtual ~Observer() = default;
7 };
8
9 class Stock {                                // subject
10     vector<Observer*> observers;
11     int price = 0;
12 public:
13     void subscribe(Observer* o) { observers.push_back(o); }
14     void unsubscribe(Observer* o) {
15         observers.erase(remove(observers.begin(), observers.end(), o),
16                             observers.end());
17     }
18     void setPrice(int p) {
19         price = p;
20         for (Observer* o : observers) o->update(price); // notify all
21     }
22 };
23
24 struct Display : Observer {
25     string name;
26     Display(string n) : name(move(n)) {}
27     void update(int price) override {
28         cout << name << " sees price " << price << "\n";
29     }
30 };
31
32 int main() {
33     Stock stock;
34     Display a("Phone"), b("Web");
35     stock.subscribe(&a); stock.subscribe(&b);
36     stock.setPrice(100); // both notified
37     stock.unsubscribe(&a);
38     stock.setPrice(120); // only Web notified
39     return 0;
40 }

```

Trade-offs

Pros. Loose coupling between subject and observers; observers added/removed at run time; supports broadcast/event architectures. **Cons.** Notification order is unspecified; risk of update storms or cycles; a dangling observer that forgot to unsubscribe causes bugs or leaks (use weak references or careful lifetime management).

Key Takeaway

Observer wires a one-to-many “notify on change” relationship behind an observer interface, the core of MVC and pub/sub. Interview keyword: “when this changes, update all those.” The pitfalls to name are lapsed-listener leaks and unspecified notification order.

2.20 State

Problem Statement. Allow an object to alter its behaviour when its internal state changes, so it appears to change its class – replacing sprawling conditionals on a state variable with a set of state objects.

When You Reach for It

The trigger is “*behaviour depends on a mode/state, and the same method does different things in each*” – a vending machine, a media player (playing/paused/stopped), a TCP connection, an order lifecycle. When methods are riddled with `if (state == ...)`, extract states into classes.

Understanding the pattern

Each state becomes a class implementing a common `State` interface; the context holds a pointer to the current state and delegates behaviour to it. A state’s methods perform the state-specific action and can transition the context to another state. Adding a new state is a new class, not another branch in every method – Open/Closed for state machines.

One Class per State, Delegate and Transition

Turn each mode into an object that knows how to behave *and* which state comes next. The context just forwards calls to its current state object. The scattered `switch(state)` logic collapses into localised, self-contained state classes.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Fan;
5  struct State {
6      virtual void pull(Fan& fan) = 0;           // the chain-pull
7      virtual string name() const = 0;
8      virtual ~State() = default;
9  };
10
11 class Fan {
12     unique_ptr<State> state;
13 public:
14     Fan(unique_ptr<State> s) : state(move(s)) {}
15     void setState(unique_ptr<State> s) { state = move(s); }
16     void pull() { state->pull(*this); }
17     string status() const { return state->name(); }
18 };
19
20 struct Off; struct Low;
21 struct Low : State {
22     void pull(Fan& fan) override;
23     string name() const override { return "Low"; }
24 };
25 struct Off : State {
26     void pull(Fan& fan) override { fan.setState(make_unique<Low>()); }
27     string name() const override { return "Off"; }
28 };
29 void Low::pull(Fan& fan) { fan.setState(make_unique<Off>()); }
30
31 int main() {
32     Fan fan(make_unique<Off>());
33     cout << fan.status() << "\n";           // Off

```

```

34     fan.pull(); cout << fan.status() << "\n";    // Low
35     fan.pull(); cout << fan.status() << "\n";    // Off
36     return 0;
37 }

```

Trade-offs

Pros. Eliminates large state conditionals; each state is isolated and testable; transitions are explicit; new states are added without editing others. **Cons.** More classes; transition logic is spread across state classes, so the overall state machine can be harder to see at a glance (a transition table can complement it).

Key Takeaway

State replaces “behaviour depends on a mode” conditionals with one class per state that both acts and decides the next state. Structurally it is Strategy, but the intent differs: State objects *swap themselves* as the machine transitions, whereas a Strategy is chosen by the client.

2.21 Strategy

Problem Statement. Define a family of interchangeable algorithms, encapsulate each one, and make them swappable at run time so the algorithm varies independently of the client that uses it.

When You Reach for It

The trigger is “*several ways to do the same thing, chosen at run time*” – different sort orders, payment methods, compression schemes, route-finding algorithms. When you catch yourself writing `if (mode == A) doA(); else doB();`, extract the modes into strategy objects.

Understanding the pattern

Encapsulate each algorithm behind a common interface. The context holds a pointer to a strategy and delegates the varying step to it, without knowing which concrete algorithm it is. Clients pick and inject a strategy; they can change it at run time. This is the Open/Closed Principle for behaviour: add a new algorithm as a new class, edit nothing existing.

Inject the Algorithm as an Object

Pull the part that varies into a strategy object and hand it to the context. The context is written once against the strategy interface; the concrete behaviour is a plug-in chosen by the client. This is composition replacing a conditional.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct PaymentStrategy {
5      virtual void pay(int amount) = 0;
6      virtual ~PaymentStrategy() = default;
7  };

```

```

8 struct CreditCard : PaymentStrategy {
9     void pay(int amt) override { cout << "Paid " << amt << " by card\n"
10    ; }
11 };
12 struct UpiPayment : PaymentStrategy {
13     void pay(int amt) override { cout << "Paid " << amt << " via UPI\n"
14    ; }
15 };
16 class Checkout {                                     // context
17     unique_ptr<PaymentStrategy> strategy;
18 public:
19     void setStrategy(unique_ptr<PaymentStrategy> s) { strategy = move(s)
20    }; }
21     void pay(int amount) { strategy->pay(amount); } // delegate
22                                     varying step
23 };
24 int main() {
25     Checkout checkout;
26     checkout.setStrategy(make_unique<CreditCard>());
27     checkout.pay(500);
28     checkout.setStrategy(make_unique<UpiPayment>()); // swap at run
29                                     time
30     checkout.pay(300);
31     return 0;
32 }

```

Trade-offs

Pros. Algorithms are interchangeable at run time; removes conditionals; each algorithm is isolated and testable; supports Open/Closed. **Cons.** Clients must know the strategies exist to choose one; more objects/classes; overkill when the behaviour never varies.

Key Takeaway

Strategy encapsulates interchangeable algorithms behind one interface and lets the client inject the choice, replacing behavioural conditionals with composition. It is the single most common behavioural pattern in interviews. Contrast with State: same shape, but the *client* picks a Strategy while a State machine swaps its own state.

2.22 Template Method

Problem Statement. Define the skeleton of an algorithm in a base-class method, deferring some steps to subclasses, so subclasses can redefine certain steps without changing the algorithm's overall structure.

When You Reach for It

The trigger is “*the same overall procedure with a few steps that differ*” – data importers that all parse/validate/save but read different formats, report generators, test set-up/tear-down. When the outline is fixed but a few steps vary, fix the outline and vary the steps.

Understanding the pattern

A base class provides a non-virtual `templateMethod()` that calls a sequence of steps in a fixed order. Some steps are concrete (shared), some are `virtual`/abstract “hooks” that subclasses override. The invariant sequence lives in one place; only the variable steps are subclassed. This inverts control: the base class calls down into the subclass (“don’t call us, we’ll call you”).

Fix the Skeleton, Override the Steps

Keep the algorithm’s order in a base method and expose just the varying steps as overridable hooks. Subclasses cannot accidentally reorder or skip the algorithm – they only fill in the blanks.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class DataProcessor {
5  public:
6      void process() {                // template method (
7          fixed order
8          read();
9          auto data = parse();
10         cout << "Processing " << data << " rows\n";
11         save();
12     }
13     virtual ~DataProcessor() = default;
14 protected:
15     virtual void read() = 0;          // varying steps
16     virtual int  parse() = 0;
17     void save() { cout << "Saved to DB\n"; } // shared step
18 };
19
20 class CsvProcessor : public DataProcessor {
21 protected:
22     void read() override { cout << "Read CSV file\n"; }
23     int  parse() override { return 42; }
24 };
25
26 int main() {
27     CsvProcessor p;
28     p.process();                // skeleton runs, steps
29     vary
30     return 0;
31 }

```

Trade-offs

Pros. Removes duplicated algorithm structure; enforces a fixed sequence; subclasses supply only what differs. **Cons.** Relies on inheritance (less flexible than composition); the base–subclass contract about which hooks to override can be subtle; changing the skeleton affects all subclasses.

Key Takeaway

Template Method locks an algorithm's structure in a base method and lets subclasses override only the variable steps – inheritance-based inversion of control. Its composition-based cousin is Strategy: Template Method varies steps via subclassing, Strategy varies the whole algorithm via an injected object.

2.23 Visitor

Problem Statement. Represent an operation to be performed on the elements of an object structure, letting you define a new operation without changing the classes of the elements it operates on.

When You Reach for It

The trigger is “*add new operations over a stable set of element types*” – an AST you want to pretty-print, type-check, and compile, without editing every node class each time. When element types rarely change but operations keep being added, Visitor centralises each operation.

Understanding the pattern

Each element exposes `accept(visitor)`, which calls back `visitor.visitConcreteType(*this)` – a technique called **double dispatch**, because the executed method depends on both the element's type and the visitor's type. A visitor bundles one operation across all element types as a set of `visit` overloads. Adding an operation is a new visitor class; the element classes are untouched.

Double Dispatch Moves Operations Out of the Elements

`accept` resolves the element's concrete type; the matching `visit` overload resolves the operation. Together they route to the right (element, operation) pair without a type switch, so operations live in visitors rather than being smeared across every element class.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Circle; struct Square;
5 struct Visitor {
6     virtual void visit(Circle&) = 0;
7     virtual void visit(Square&) = 0;
8     virtual ~Visitor() = default;
9 };
10
11 struct Shape { virtual void accept(Visitor& v) = 0; virtual ~Shape() =
    default; };
12 struct Circle : Shape {
13     double r = 2;
14     void accept(Visitor& v) override { v.visit(*this); } // double
        dispatch
15 };
16 struct Square : Shape {

```

```

17     double side = 3;
18     void accept(Visitor& v) override { v.visit(*this); }
19 };
20
21 struct AreaVisitor : Visitor { // a new operation, no
    shape edits
22     void visit(Circle& c) override { cout << "Circle area " << 3.14159
        * c.r * c.r << "\n"; }
23     void visit(Square& s) override { cout << "Square area " << s.side *
        s.side << "\n"; }
24 };
25
26 int main() {
27     vector<unique_ptr<Shape>> shapes;
28     shapes.push_back(make_unique<Circle>());
29     shapes.push_back(make_unique<Square>());
30     AreaVisitor area;
31     for (auto& s : shapes) s->accept(area); // right visit() per
        type
32     return 0;
33 }

```

Trade-offs

Pros. Add new operations easily (one new visitor); gathers a scattered operation into one class; keeps element classes clean. **Cons.** Adding a new *element type* forces editing every visitor – the exact opposite trade-off to most patterns. Use only when element types are stable and operations grow.

Key Takeaway

Visitor externalises operations over an object structure via double dispatch, so new operations are new visitor classes with the elements untouched – at the cost of making new element types expensive. It is the inverse trade-off of a type switch, and the go-to for AST/compiler passes.

Chapter 3

LLD Case Studies

An LLD interview is a conversation, not a coding sprint. You are judged on how you clarify vague requirements, carve the problem into classes with clean responsibilities, choose patterns with justification, and leave room for the features the interviewer will ask for next. Each case study here follows that arc: clarify, identify entities, design, and name the patterns that keep the design flexible.

3.1 How to Approach Any LLD Problem

Before writing a single class, run the same four steps every time. They turn a scary open-ended prompt into a structured design.

The Four-Step LLD Method

1. **Clarify & scope.** Ask about requirements and constraints; state your assumptions. Nail down what is in and out of scope *before* designing.
2. **Identify entities.** Extract the nouns – they become classes; the verbs become methods. Assign each class one clear responsibility (SRP).
3. **Design relationships & patterns.** Decide how classes relate (composition over inheritance) and where a known pattern fits (Strategy for varying rules, Factory for creation, Observer for notifications, State for lifecycles).
4. **Walk a scenario & extend.** Trace one end-to-end flow, then show how a likely new requirement slots in without rewrites.

Patterns That Recur Across Case Studies

- Pricing/matching/eligibility rules that vary – **Strategy**.
- Creating objects by type – **Factory**.
- Lifecycle with modes (order, game, machine) – **State**.
- “Notify these when that happens” – **Observer**.
- One shared resource (config, id generator, in-memory store) – **Singleton** (used judiciously).
- Complex object assembly – **Builder**.

The case studies progress from the classic “design a parking lot” to notification, logging, and rate-limiting systems. Each names the patterns from Chapter 2 it relies on, so the two chapters reinforce each other.

3.2 Design a Parking Lot

Problem Statement. Design a parking lot with multiple levels, multiple spot sizes (motorcycle, compact, large), vehicle entry/exit, ticketing, and fee calculation. It should assign the smallest spot a vehicle fits in and compute the fee on exit.

Clarify Before Designing

Ask: How many levels and spot types? Can a large vehicle occupy multiple spots? Is pricing flat, hourly, or tiered? Multiple entry/exit gates? Payment methods? For this design assume: several levels, three spot sizes, one spot per vehicle, hourly pricing that may vary by vehicle type, and multiple gates.

Identify the entities

The nouns give the classes: **ParkingLot** (owns levels), **Level** (owns spots), **ParkingSpot** (has a size and occupancy), **Vehicle** (abstract; **Motorcycle**, **Car**, **Truck**), **Ticket** (issued at entry), and a **FeeStrategy** for pricing. The verbs – park, unpark, pay – become methods.

Patterns That Keep It Flexible

Strategy for **FeeStrategy** so pricing rules change without touching parking logic. **Factory** to create the right **Vehicle**/spot type. A **Singleton** (or injected single instance) **ParkingLot** as the entry point. Spot assignment is a policy method, easy to swap (nearest, smallest-fit).

Class design (C++ skeleton)

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  enum class SpotSize { Motorcycle, Compact, Large };
5
6  class Vehicle {
7  public:
8      string plate;
9      SpotSize required;
10     Vehicle(string p, SpotSize s) : plate(move(p)), required(s) {}
11     virtual ~Vehicle() = default;
12 };
13 struct Car : Vehicle { Car(string p) : Vehicle(move(p), SpotSize::
    Compact) {} };
14 struct Bike : Vehicle { Bike(string p) : Vehicle(move(p), SpotSize::
    Motorcycle) {} };
15
16 struct FeeStrategy {                                // Strategy: pricing
17     varies
18     virtual double fee(int hours) const = 0;
19     virtual ~FeeStrategy() = default;
20 };
21 struct FlatHourly : FeeStrategy {
22     double rate; FlatHourly(double r) : rate(r) {}
23     double fee(int hours) const override { return rate * hours; }
24 };

```

```

25 class ParkingSpot {
26 public:
27     int id; SpotSize size; Vehicle* occupant = nullptr;
28     ParkingSpot(int i, SpotSize s) : id(i), size(s) {}
29     bool canFit(const Vehicle& v) const {
30         return occupant == nullptr && size >= v.required;    // smallest
31         -fit rule
32     };
33
34 struct Ticket { int spotId; string plate; time_t entry; };
35
36 class Level {
37     vector<ParkingSpot> spots;
38 public:
39     Level(int n) { for (int i = 0; i < n; i++) spots.emplace_back(i,
40         SpotSize::Compact); }
41     ParkingSpot* findSpot(const Vehicle& v) {
42         for (auto& s : spots) if (s.canFit(v)) return &s;    //
43         assignment policy
44         return nullptr;
45     };
46
47 class ParkingLot {
48     vector<Level> levels;
49     unique_ptr<FeeStrategy> pricing;
50 public:
51     ParkingLot(unique_ptr<FeeStrategy> p) : pricing(move(p)) {}
52     void addLevel(int spots) { levels.emplace_back(spots); }
53
54     Ticket* park(Vehicle& v) {
55         for (auto& level : levels) {
56             if (auto* spot = level.findSpot(v)) {
57                 spot->occupant = &v;
58                 return new Ticket{ spot->id, v.plate, time(nullptr) };
59             }
60             return nullptr;    // lot full
61         }
62         double unpark(Ticket* t, int hours) { return pricing->fee(hours); }
63     };
64
65 int main() {
66     ParkingLot lot(make_unique<FlatHourly>(20.0));
67     lot.addLevel(2);
68     Car car("KA-01-1234");
69     Ticket* t = lot.park(car);
70     cout << (t ? "Parked, fee for 3h = " : "Full ") ;
71     if (t) cout << lot.unpark(t, 3) << "\n";
72     return 0;
73 }

```

Key Takeaway

Parking lot is the canonical LLD warm-up. Score points by: clarifying scope first, giving each class one job (SRP), isolating pricing behind a **Strategy**, and making spot-assignment

a swappable policy. Likely follow-ups: multiple gates (concurrency, locking a spot atomically), payment methods (another Strategy), and reservations.

3.3 Design an Elevator System

Problem Statement. Design the control system for a building with multiple elevators. Handle external hall calls (up/down on a floor) and internal car requests (go to floor N), decide which elevator serves a request, and move each car efficiently.

Clarify Before Designing

Ask: How many elevators and floors? What scheduling goal – minimise wait, minimise travel, or fairness? Are there express/service elevators? For this design assume: N elevators, a central dispatcher, a nearest-suitable-car scheduling policy that is swappable, and each car running a SCAN (elevator algorithm) sweep.

Identify the entities

Elevator (a car with current floor, direction, and its set of target floors), **Request** (hall or car call), **Direction** (up/down/idle), a **Dispatcher** that assigns requests to cars via a **SchedulingStrategy**, and a **Building** owning the elevators. Each elevator behaves as a small state machine: idle, moving up, moving down.

Patterns That Keep It Flexible

Strategy for **SchedulingStrategy** (nearest car, least load, zoning) so dispatch policy is a plug-in. **State** for each elevator's motion (idle/up/down) to keep the movement logic clean. Optionally **Observer** so displays update as a car moves. The car's own sweep is the classic SCAN scheduling of its pending stops.

Class design (C++ skeleton)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 enum class Direction { Up, Down, Idle };
5
6 class Elevator {
7 public:
8     int id, floor = 0;
9     Direction dir = Direction::Idle;
10    set<int> targets; // pending stops
11
12    void addTarget(int f) { targets.insert(f); }
13    int distanceTo(int f) const { return abs(floor - f); }
14
15    void step() { // one SCAN move
16        if (targets.empty()) { dir = Direction::Idle; return; }
17        int next = (dir == Direction::Down)
18            ? *prev(targets.upper_bound(floor))
19            : *targets.lower_bound(floor);
20        if (next > floor) { floor++; dir = Direction::Up; }
21        else if (next < floor) { floor--; dir = Direction::Down; }
    }
};

```

```

22         else { targets.erase(floor); cout << "Car " << id << " stop @ "
23             << floor << "\n"; }
24     };
25
26     struct SchedulingStrategy {                                // Strategy: which car
27         serves                                                 serves
28         virtual Elevator* choose(vector<Elevator>& cars, int floor) = 0;
29         virtual ~SchedulingStrategy() = default;
30     };
31     struct NearestCar : SchedulingStrategy {
32         Elevator* choose(vector<Elevator>& cars, int floor) override {
33             Elevator* best = nullptr;
34             for (auto& c : cars)
35                 if (!best || c.distanceTo(floor) < best->distanceTo(floor))
36                     best = &c;
37             return best;
38         }
39     };
40
41     class Dispatcher {
42     public:
43         Dispatcher(int n, unique_ptr<SchedulingStrategy> p) : policy(move(p)) {
44             for (int i = 0; i < n; i++) cars.push_back({i});
45         }
46         void hallCall(int floor) {
47             Elevator* car = policy->choose(cars, floor);
48             car->addTarget(floor);
49             cout << "Assigned car " << car->id << " to floor " << floor <<
50                 "\n";
51         }
52         void tick() { for (auto& c : cars) c.step(); }
53     };
54
55     int main() {
56         Dispatcher d(2, make_unique<NearestCar>());
57         d.hallCall(5);
58         for (int i = 0; i < 6; i++) d.tick();
59         return 0;
60     }

```

Key Takeaway

Elevator design shines when you separate *dispatch* (which car, a **Strategy**) from *motion* (how a car sweeps its stops, a **State**/SCAN algorithm). Discuss the scheduling goal explicitly. Follow-ups: concurrency across cars, priority/express calls, and starvation avoidance.

3.4 Design a Vending Machine

Problem Statement. Design a vending machine that holds products in slots, accepts coins/notes, dispenses a selected product, returns change, and handles out-of-stock and insufficient-funds cases.

Clarify Before Designing

Ask: What payment types (cash, card)? Should it make exact change, and what if it cannot? Can a selection be cancelled mid-transaction? For this design assume: cash only, refunds allowed, and a clear lifecycle from idle to dispensing. The lifecycle is the heart of the problem.

Identify the entities

VendingMachine (the context/state machine), **Product**, **Slot/Inventory** (stock levels), **Coin**/payment handling, and a set of **State** classes – **IdleState**, **HasMoneyState**, **DispensingState**. The machine's behaviour for the same action ("insert coin," "select") differs entirely by state, which is the signal for the State pattern.

State Pattern Is the Backbone

Model the machine as a **State** machine: each state handles **insertCoin**, **selectProduct**, and **dispense** appropriately and transitions the machine forward. This replaces a giant **switch(currentState)** in every method and makes illegal transitions (dispensing before paying) structurally impossible.

Class design (C++ skeleton)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class VendingMachine;
5 struct State {
6     virtual void insertCoin(VendingMachine&, int amount) = 0;
7     virtual void selectProduct(VendingMachine&, const string& code) =
8         0;
9     virtual ~State() = default;
10 };
11
12 class VendingMachine {
13     unique_ptr<State> state;
14 public:
15     int balance = 0;
16     unordered_map<string, int> stock;           // code -> quantity
17     unordered_map<string, int> price;           // code -> price
18
19     VendingMachine();                          // starts in IdleState
20     void setState(unique_ptr<State> s) { state = move(s); }
21     void insertCoin(int a) { state->insertCoin(*this, a); }
22     void selectProduct(const string& c) { state->selectProduct(*this, c
23         ); }
24 };
25
26 struct IdleState : State {
27     void insertCoin(VendingMachine& m, int a) override;
28     void selectProduct(VendingMachine&, const string&) override {
29         cout << "Insert money first\n";
30     }
31 };
32
33 struct HasMoneyState : State {

```

```

31     void insertCoin(VendingMachine& m, int a) override { m.balance += a
        ; }
32     void selectProduct(VendingMachine& m, const string& code) override
        {
33         if (m.stock[code] == 0) { cout << "Out of stock\n"; return; }
34         if (m.balance < m.price[code]) { cout << "Insufficient funds\n"
            ; return; }
35         m.stock[code]--;
36         int change = m.balance - m.price[code];
37         m.balance = 0;
38         cout << "Dispensed " << code << ", change " << change << "\n";
39         m.setState(make_unique<IdleState>()); // back to idle
40     }
41 };
42
43 void IdleState::insertCoin(VendingMachine& m, int a) {
44     m.balance += a;
45     m.setState(make_unique<HasMoneyState>()); // transition
46 }
47 VendingMachine::VendingMachine() : state(make_unique<IdleState>()) {}
48
49 int main() {
50     VendingMachine m;
51     m.stock["A1"] = 2; m.price["A1"] = 30;
52     m.selectProduct("A1"); // "Insert money first"
53     m.insertCoin(20); m.insertCoin(20);
54     m.selectProduct("A1"); // dispenses, change 10
55     return 0;
56 }

```

Key Takeaway

The vending machine is the textbook **State** pattern problem: recognise that the same operations behave differently per mode and give each mode a class that acts and transitions. Follow-ups: card payments (Strategy), exact-change detection, and cancel/refund as another transition.

3.5 Design Tic-Tac-Toe

Problem Statement. Design a Tic-Tac-Toe game (generalisable to an $n \times n$ board) supporting two players taking turns, move validation, and win/draw detection. Keep the design extensible to other grid games.

Clarify Before Designing

Ask: Fixed 3×3 or general $n \times n$ with k -in-a-row to win? Two human players or human-vs-AI? For this design assume: general board size, n -in-a-row to win, two players, and an efficient $O(1)$ win check per move rather than rescanning the board.

Identify the entities

Game (orchestrates turns and end conditions), **Board** (grid state and win detection), **Player** (has a mark/symbol), **Move**, and **Piece/Symbol**. The game is a small lifecycle: in-progress, won, drawn – a light State/enum.

Incremental Win Check in $O(1)$ per Move

Instead of scanning all lines after every move, keep running counts per row, per column, and the two diagonals for each player. A move updates just its row, column, and (if on a diagonal) the diagonal counters; a count reaching n means a win. This turns win detection from $O(n^2)$ into $O(1)$ per move.

Class design (C++ skeleton)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Board {
5     int n;
6     vector<vector<char>> grid;
7     vector<int> rowCnt, colCnt;           // signed counts per
        player
8     int diag = 0, anti = 0;
9 public:
10     Board(int size) : n(size), grid(size, vector<char>(size, '.')),
11                     rowCnt(size, 0), colCnt(size, 0) {}
12
13     bool inBounds(int r, int c) const { return r >= 0 && r < n && c >=
14         0 && c < n; }
15     bool isFree(int r, int c) const { return grid[r][c] == '.'; }
16
17     // returns true if this move wins. player = +1 (X) or -1 (O)
18     bool place(int r, int c, char mark, int player) {
19         grid[r][c] = mark;
20         rowCnt[r] += player; colCnt[c] += player;
21         if (r == c) diag += player;
22         if (r + c == n - 1) anti += player;
23         int win = n * player;
24         return rowCnt[r] == win || colCnt[c] == win || diag == win ||
25             anti == win;
26     }
27 };
28
29 struct Player { string name; char mark; int id; }; // id = +1 or -1
30
31 class Game {
32     Board board; vector<Player> players; int turn = 0; int movesLeft;
33 public:
34     Game(int n, Player a, Player b)
35         : board(n), players{a, b}, movesLeft(n * n) {}
36
37     string move(int r, int c) {
38         Player& p = players[turn];
39         if (!board.inBounds(r, c) || !board.isFree(r, c)) return "
40             Invalid move";
41         bool won = board.place(r, c, p.mark, p.id);
42         movesLeft--;
43         if (won) return p.name + " wins";
44         if (movesLeft == 0) return "Draw";
45         turn ^= 1;           // next player
46         return "ok";
47     }
48 }

```

```

44     }
45 };
46
47 int main() {
48     Game g(3, {"Alice", 'X', +1}, {"Bob", 'O', -1});
49     cout << g.move(0,0) << "\n";           // X
50     cout << g.move(1,0) << "\n";           // O
51     cout << g.move(0,1) << "\n";           // X
52     cout << g.move(1,1) << "\n";           // O
53     cout << g.move(0,2) << "\n";           // X wins (top row)
54     return 0;
55 }

```

Key Takeaway

Tic-Tac-Toe rewards two things: a clean separation of **Game** (rules/turns) from **Board** (state/win check), and the $O(1)$ incremental win-detection insight. Generalise to $n \times n$ up front. Follow-ups: an AI player via **Strategy** (minimax), and other grid games (Connect Four) reusing the board abstraction.

3.6 Design Splitwise (Expense Sharing)

Problem Statement. Design an expense-sharing app: users form groups, add expenses split among members (equally, by exact amount, or by percentage), and query who owes whom. Support simplifying debts so the fewest transactions settle everyone.

Clarify Before Designing

Ask: What split types – equal, exact, percentage, shares? Do we track pairwise balances or a single net balance per user? Is debt simplification required? For this design assume: all three split types, a net-balance model, and an optional simplify step.

Identify the entities

User, Group, Expense (amount, payer, participants, and a Split), a SplitStrategy (Equal, Exact, Percent), and a BalanceSheet/ExpenseManager that maintains net balances. The varying part – how an amount is divided – is a textbook Strategy.

Split Logic Is a Strategy; Balances Are Netted

Model each split rule as a **SplitStrategy** producing a per-participant owed amount, so new rules never touch the manager. Store a single net balance per (debtor, creditor) or per user; adding an expense just adjusts these. Debt simplification is then a graph settle: net each person, then greedily match biggest creditor with biggest debtor.

Class design (C++ skeleton)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SplitStrategy {                               // Strategy: division
5     rule
6     // returns each participant's share of 'amount'

```

```

6     virtual vector<double> split(double amount, int n,
7                                   const vector<double>& params) const =
8                                   0;
9     virtual ~SplitStrategy() = default;
10 };
11 struct EqualSplit : SplitStrategy {
12     vector<double> split(double amount, int n, const vector<double>&)
13     const override {
14         return vector<double>(n, amount / n);
15     }
16 };
17 struct PercentSplit : SplitStrategy {
18     vector<double> split(double amount, int n, const vector<double>&
19     pct) const override {
20         vector<double> out(n);
21         for (int i = 0; i < n; i++) out[i] = amount * pct[i] / 100.0;
22         return out;
23     }
24 };
25
26 class ExpenseManager {
27     // balance[a][b] = how much a owes b
28     map<string, map<string, double>> balance;
29 public:
30     void addExpense(const string& payer, const vector<string>& members,
31                    double amount, const SplitStrategy& strat,
32                    const vector<double>& params = {}) {
33         auto shares = strat.split(amount, members.size(), params);
34         for (size_t i = 0; i < members.size(); i++) {
35             if (members[i] == payer) continue;
36             balance[members[i]][payer] += shares[i]; // member owes
37             payer
38         }
39     }
40     void showBalances() {
41         for (auto& [debtor, m] : balance)
42             for (auto& [creditor, amt] : m)
43                 if (amt > 1e-9)
44                     cout << debtor << " owes " << creditor << ": " <<
45                     amt << "\n";
46     }
47 };
48
49 int main() {
50     ExpenseManager mgr;
51     vector<string> group = {"Alice", "Bob", "Carol"};
52     mgr.addExpense("Alice", group, 300, EqualSplit{}); // each owes
53     Alice 100
54     mgr.showBalances();
55     return 0;
56 }

```

Key Takeaway

Splitwise is a Strategy showcase: isolate the split rule so equal/exact/ percent are plug-ins, and keep a simple netted balance model. The memorable extension is *debt simplification* – reduce the balance graph to the minimum number of settling transactions by greedily

matching the largest creditor and debtor.

3.7 Design a Notification System

Problem Statement. Design a system that sends notifications to users across multiple channels (email, SMS, push), lets users set channel preferences, supports templated messages, and can add new channels without disturbing existing code.

Clarify Before Designing

Ask: Which channels, and can more be added later? Are messages templated? Do users choose channels per notification type? Synchronous or queued? For this design assume: pluggable channels, per-user preferences, templated content, and asynchronous-friendly dispatch.

Identify the entities

A `NotificationChannel` interface (Email, SMS, Push), a `ChannelFactory` to create them, a `NotificationService` that orchestrates sending, `User` preferences, and optionally `Observer` so subscribers of an event are notified. Message building can use a **Builder** or template engine.

Channels Are Strategies Behind a Factory; Events Fan Out via Observer

Each channel implements a common `send` interface (a **Strategy**/plug-in); a **Factory** maps a channel name to its implementation, so a new channel is one new class plus a registry entry. For “notify everyone interested in event X,” layer **Observer** on top: subscribers register, the event publishes, each is sent via their preferred channel.

Class design (C++ skeleton)

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct NotificationChannel {                                // Strategy per channel
5      virtual void send(const string& to, const string& msg) = 0;
6      virtual ~NotificationChannel() = default;
7  };
8  struct EmailChannel : NotificationChannel {
9      void send(const string& to, const string& msg) override {
10         cout << "Email to " << to << ": " << msg << "\n";
11     }
12 };
13 struct SmsChannel : NotificationChannel {
14     void send(const string& to, const string& msg) override {
15         cout << "SMS to " << to << ": " << msg << "\n";
16     }
17 };
18
19 struct ChannelFactory {                                    // Factory: name ->
20     channel
21     static unique_ptr<NotificationChannel> create(const string& type) {
22         if (type == "email") return make_unique<EmailChannel>();
23         if (type == "sms")   return make_unique<SmsChannel>();

```

```

23         return nullptr;
24     }
25 };
26
27 struct User { string id; vector<string> preferredChannels; string
    contact; };
28
29 class NotificationService {
30 public:
31     void notify(const User& user, const string& message) {
32         for (const auto& ch : user.preferredChannels) {
33             auto channel = ChannelFactory::create(ch);
34             if (channel) channel->send(user.contact, message);    // fan
                           out
35         }
36     }
37 };
38
39 int main() {
40     NotificationService service;
41     User u{"u1", {"email", "sms"}, "alice@x.com"};
42     service.notify(u, "Your order shipped");    // sent on both
        channels
43     return 0;
44 }

```

Key Takeaway

A notification system is a clean **Strategy + Factory** design: channels behind one interface, created by name, so new channels never touch dispatch. Add **Observer** for event-driven fan-out and **Decorator** for cross-cutting concerns (retry, rate-limit, encryption) around a channel. Discuss queuing for reliability at scale.

3.8 Design a Logging Framework

Problem Statement. Design a logging framework supporting severity levels (DEBUG, INFO, WARN, ERROR), multiple output sinks (console, file, network), a configurable minimum level, and the ability to add sinks or formats without changing call sites.

Clarify Before Designing

Ask: What levels and can they be filtered globally or per sink? Multiple simultaneous sinks? Thread-safe? Should a message be passed along a chain of level handlers or broadcast to all sinks? For this design assume: standard levels, multiple sinks, a global threshold, and thread safety as a discussion point.

Identify the entities

A **Logger** (front door, often a **Singleton**), a **LogLevel** enum, **LogSink/Appender** implementations (**ConsoleSink**, **FileSink**), and a **Formatter**. Two classic pattern framings apply: broadcast to all sinks (**Observer**) or pass up a **Chain of Responsibility** of level handlers.

Level Filtering + Pluggable Sinks

Keep a single global threshold so sub-threshold messages are dropped cheaply. Model each output as a LogSink plug-in the logger writes to – adding a sink never touches call sites. A **Chain of Responsibility** variant instead routes a message through DEBUG→INFO→WARN→ERROR handlers, each deciding whether to act; both are valid, name the trade-off.

Class design (C++ skeleton)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 enum class Level { Debug, Info, Warn, Error };
5
6 struct LogSink {                                     // pluggable output
7     virtual void write(Level lvl, const string& msg) = 0;
8     virtual ~LogSink() = default;
9 };
10 struct ConsoleSink : LogSink {
11     void write(Level, const string& msg) override { cout << "[console] "
12         << msg << "\n"; }
13 };
14 struct FileSink : LogSink {
15     void write(Level, const string& msg) override { cout << "[file] "
16         << msg << "\n"; }
17 };
18
19 class Logger {                                       // Singleton front door
20     Level threshold = Level::Info;
21     vector<unique_ptr<LogSink>> sinks;
22     Logger() = default;
23 public:
24     static Logger& instance() { static Logger l; return l; }
25     void setThreshold(Level l) { threshold = l; }
26     void addSink(unique_ptr<LogSink> s) { sinks.push_back(move(s)); }
27
28     void log(Level lvl, const string& msg) {
29         if (lvl < threshold) return;                // cheap filter
30         for (auto& s : sinks) s->write(lvl, msg);    // broadcast to
31             sinks
32     }
33 };
34
35 int main() {
36     Logger& log = Logger::instance();
37     log.addSink(make_unique<ConsoleSink>());
38     log.addSink(make_unique<FileSink>());
39     log.setThreshold(Level::Info);
40     log.log(Level::Debug, "verbose");                // filtered out
41     log.log(Level::Error, "something broke");        // goes to both sinks
42     return 0;
43 }

```

Key Takeaway

A logger design lets you name several patterns naturally: **Singleton** for the global logger, pluggable **Sinks** (Strategy/Observer) for outputs, and optionally a **Chain of Responsibility** over levels. The must-mention non-functional concern is *thread safety* – a lock or a lock-free queue around the sink writes at scale.

3.9 Design a Rate Limiter

Problem Statement. Design a rate limiter that caps how many requests a client may make in a time window – e.g. 100 requests per minute – and lets the algorithm (token bucket, sliding window, fixed window) be swapped. Return allow/deny per request.

Clarify Before Designing

Ask: Per-user, per-IP, or global limits? Which algorithm – fixed window, sliding window log/counter, token bucket, leaky bucket? Distributed across servers or single-node? For this design assume: per-client keys, a swappable algorithm, and single-node with a note on distributed extensions.

Identify the entities

A `RateLimiter` facade keyed by client, and a `RateLimitStrategy` interface with implementations (`TokenBucket`, `FixedWindow`, `SlidingWindow`). Each strategy stores per-client state and answers `allow()`. The varying algorithm is, again, a Strategy.

Token Bucket in a Nutshell

A token bucket holds up to **capacity** tokens, refilled at a steady **rate** per second. Each request tries to take one token: succeed and proceed, or fail and get rejected. It naturally allows short bursts (up to capacity) while bounding the long-run average – which is why it is the most common choice. Refill lazily by computing elapsed time on each call rather than running a timer.

Class design (C++ skeleton)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct RateLimitStrategy {
5     virtual bool allow(const string& client) = 0;
6     virtual ~RateLimitStrategy() = default;
7 };
8
9 class TokenBucket : public RateLimitStrategy {
10     double capacity, refillPerSec;
11     struct Bucket { double tokens; double lastRefill; };
12     unordered_map<string, Bucket> buckets;
13
14     double now() {
15         return chrono::duration<double>{(
16             chrono::steady_clock::now().time_since_epoch()).count()};
17     }
18 public:

```

```

19     TokenBucket(double cap, double rate) : capacity(cap), refillPerSec(
20         rate) {}
21     bool allow(const string& client) override {
22         double t = now();
23         auto& b = buckets.try_emplace(client, Bucket{capacity, t}).
24             first->second;
25         b.tokens = min(capacity, b.tokens + (t - b.lastRefill) *
26             refillPerSec);
27         b.lastRefill = t;
28         if (b.tokens >= 1.0) { b.tokens -= 1.0; return true; } //
29             consume
30         return false; //
31             throttled
32     }
33 };
34
35 class RateLimiter { // facade over the
36     strategy
37     unique_ptr<RateLimitStrategy> strategy;
38 public:
39     RateLimiter(unique_ptr<RateLimitStrategy> s) : strategy(move(s)) {}
40     bool allow(const string& client) { return strategy->allow(client);
41     }
42 };
43
44 int main() {
45     RateLimiter limiter(make_unique<TokenBucket>(3, 1)); // burst 3,
46         1/s
47     for (int i = 0; i < 5; i++)
48         cout << "req " << i << ": " << (limiter.allow("user1") ? "OK" :
49             "DENY") << "\n";
50     return 0;
51 }

```

Key Takeaway

Rate limiting is a **Strategy** problem: token bucket, fixed window, and sliding window are interchangeable behind one `allow()` interface. Know token bucket's burst-vs-average trade-off cold, and compute refill lazily. The senior-level extension is *distributed* limiting – shared counters in Redis with atomic increments and expiry.

Chapter 4

CS Fundamentals: Interview Q&A

The technical interview and written test almost always include core-subject questions: object orientation, databases, operating systems, and networks. This chapter is a fast, answer-first revision of the questions asked most often – each answer kept to the crisp, correct level an interviewer wants to hear, not a textbook chapter.

4.1 How to Use This Chapter

Each section is a run of question-and-answer pairs grouped by subject. The answers are deliberately concise: state the definition, the key distinction, and one example or “why it matters,” which is exactly the shape of a strong verbal answer. Read them as flashcards – cover the answer, attempt it aloud, then check.

Answer Structure That Impresses

For any “what is X / difference between X and Y” question, answer in three beats: (1) a one-sentence definition, (2) the crucial distinction or property, (3) a concrete example or consequence. Rambling loses points; this three-beat shape signals clarity.

4.2 Object-Oriented Programming

Core concepts

Q. What are the four pillars of OOP?

A. Encapsulation (bundle data with methods and hide the data), abstraction (expose *what*, hide *how*), inheritance (reuse and specialise via an “is-a” relationship), and polymorphism (one interface, many run-time behaviours). See Section 1.2.

Q. Difference between abstraction and encapsulation?

A. Abstraction hides *complexity/implementation* – you see a simple interface. Encapsulation hides *data* – fields are private and guarded by methods. Abstraction is a design-level idea; encapsulation is its language-level enforcement. They usually appear together but answer different questions: “what do I expose?” vs. “how do I protect it?”

Q. Difference between overloading and overriding?

A. Overloading is *compile-time* polymorphism: same method name, different parameter lists, resolved by the compiler. Overriding is *run-time* polymorphism: a subclass redefines a base class’s `virtual` method with the same signature, resolved by the object’s actual type via the vtable.

Q. What is a virtual function and how is run-time polymorphism implemented?

A. A `virtual` function can be overridden and is dispatched on the object’s dynamic type. In C++ this uses a per-class *vtable* (a table of function pointers) and a per-object *vptr* pointing to it; calling a virtual function follows the *vptr* to the correct override at run time.

Q. What is a pure virtual function / abstract class?

A. A pure virtual function (`= 0`) has no body and must be overridden. A class with at least one is *abstract* – it cannot be instantiated and serves as an interface. It is how C++ expresses “this is a contract, not a concrete type.”

Inheritance and relationships

Q. Compile-time vs run-time polymorphism – which is faster and why?

A. Compile-time (overloading, templates) is faster: the call is resolved and can be inlined at compile time, with no indirection. Run-time (virtual calls) costs one pointer indirection through the vtable and blocks inlining, but buys flexibility. Prefer static dispatch when the type is known.

Q. Why favour composition over inheritance?

A. Inheritance is rigid – it fixes behaviour at compile time, exposes the base class’s internals to subclasses, and can violate encapsulation. Composition (has-a) lets you change behaviour at run time by swapping a member and keeps classes loosely coupled. Use inheritance only for a true, stable “is-a.”

Q. What is the diamond problem?

A. In multiple inheritance, if class D inherits from B and C which both inherit from A, D can get two copies of A’s members, causing ambiguity. C++ solves it with *virtual inheritance*, which ensures a single shared A subobject. Many languages avoid it by disallowing multiple class inheritance.

Q. Difference between an abstract class and an interface?

A. An abstract class can hold state and concrete methods alongside abstract ones and models an “is-a” with shared implementation. An interface (in C++, a class of only pure virtual functions)

is a pure contract with no state. A class typically extends one abstract class but implements many interfaces.

Applied

Q. What are the SOLID principles?

A. Single Responsibility (one reason to change), Open/Closed (extend without modifying), Liskov Substitution (subtypes substitutable for their base), Interface Segregation (small focused interfaces), Dependency Inversion (depend on abstractions). Detailed in Section 1.4.

Q. What is dependency injection and why is it useful?

A. Instead of a class constructing its own dependencies, they are passed in (via constructor or setter). This inverts control, decouples the class from concrete implementations, and makes it testable – you can inject a mock. It is the practical face of the Dependency Inversion Principle.

Q. Shallow copy vs deep copy?

A. A shallow copy duplicates the object's fields as-is, so pointer members still point to the *same* underlying data – two objects share it. A deep copy also duplicates what the pointers reference, giving fully independent objects. Deep copy matters whenever an object owns heap memory or other resources.

Q. What is the difference between a class and an object?

A. A class is a blueprint – it defines fields and methods. An object is a concrete instance of that blueprint, occupying memory with its own field values. One class, many objects.

4.3 Database Management Systems

Foundations

Q. What is a DBMS and what problems does it solve over flat files?

A. A DBMS manages structured data with querying, concurrency control, integrity constraints, transactions, and recovery. Over flat files it adds data independence, controlled redundancy, multi-user safety (ACID), and a declarative query language, removing the need for each application to reimplement these.

Q. Difference between primary key, candidate key, and foreign key?

A. A candidate key is any minimal set of columns uniquely identifying a row. The primary key is the chosen candidate key (unique, non-null). A foreign key is a column referencing another table's primary key, enforcing referential integrity between tables.

Q. What is normalization? Explain 1NF, 2NF, 3NF.

A. Normalization organises columns to reduce redundancy and anomalies. **1NF**: atomic values, no repeating groups. **2NF**: 1NF plus no partial dependency of a non-key column on part of a composite key. **3NF**: 2NF plus no transitive dependency (non-key columns depend only on the key). BCNF is a stricter 3NF.

Q. What is denormalization and when is it used?

A. Deliberately introducing redundancy (e.g. storing a computed or duplicated column) to speed up reads by avoiding joins. Used in read-heavy/analytics systems where query performance outweighs the cost of maintaining consistency on writes.

Transactions and concurrency

Q. What are ACID properties?

A. **Atomicity** (all-or-nothing), **Consistency** (a transaction moves the DB from one valid state to another), **Isolation** (concurrent transactions don't interfere as if serial), **Durability** (committed changes survive crashes). Together they make transactions reliable.

Q. What are the SQL isolation levels and what anomalies do they prevent?

A. From weakest to strongest: *Read Uncommitted* (allows dirty reads), *Read Committed* (prevents dirty reads), *Repeatable Read* (also prevents non-repeatable reads), *Serializable* (also prevents phantom reads). Higher isolation means more correctness but less concurrency.

Q. What is a deadlock and how do databases handle it?

A. Two or more transactions each wait for locks the others hold, forming a cycle so none proceed. DBMSs detect it (wait-for graph) and abort a victim, or prevent it via ordering/timeouts. It is the transactional analogue of an OS deadlock.

Q. Difference between optimistic and pessimistic locking?

A. Pessimistic locking locks data before use, assuming conflicts are likely – safe but reduces concurrency. Optimistic locking assumes conflicts are rare: it checks at commit (via a version/-timestamp) whether data changed, and retries if so. Optimistic suits low-contention, read-heavy workloads.

Indexing and querying

Q. What is an index and what is the trade-off?

A. An index is an auxiliary structure (usually a B+ tree) that speeds up lookups and range queries on indexed columns, turning a full scan into a logarithmic search. The trade-off: extra storage and slower writes/updates, since indexes must be maintained.

Q. Clustered vs non-clustered index?

A. A clustered index determines the physical order of rows – there is at most one per table, and the leaf level *is* the data. A non-clustered index is separate and stores pointers (or the clustered key) to the rows; a table can have many. Clustered lookups avoid an extra indirection.

Q. What is the difference between WHERE and HAVING?

A. **WHERE** filters rows *before* grouping; **HAVING** filters groups *after* **GROUP BY** aggregation. You use **HAVING** to filter on aggregate results like `COUNT(*) > 5`.

Q. Difference between DELETE, TRUNCATE, and DROP?

A. **DELETE** removes rows (optionally with a **WHERE**), is logged, and can be rolled back. **TRUNCATE** removes all rows quickly with minimal logging and resets identity, but is harder to roll back. **DROP** removes the entire table structure and data.

Scale

Q. SQL vs NoSQL – when would you choose each?

A. SQL (relational) suits structured data, complex joins, and strong consistency/ACID – e.g. finance. NoSQL (document, key-value, column, graph) suits flexible schemas, huge scale, and high write throughput with eventual consistency – e.g. user feeds, telemetry. Choose by consistency needs, schema stability, and access patterns.

Q. What is the CAP theorem?

A. In a distributed store you cannot simultaneously guarantee all of Consistency, Availability, and Partition tolerance; under a network partition you must trade consistency for availability or vice versa. Real systems (partition tolerance being mandatory) are effectively CP or AP.

4.4 Operating Systems

Processes and threads

Q. Difference between a process and a thread?

A. A process is an independent program in execution with its own address space, memory, and resources. A thread is a lightweight unit of execution *within* a process; threads of one process share its address space and resources but have their own stack and registers. Threads are cheaper to create and switch, but a bug in one can corrupt the shared space.

Q. What is a context switch and why is it expensive?

A. Saving the state (registers, program counter) of the currently running process/thread and loading another's, so the CPU can switch execution. It is costly because of the save/restore work plus cache and TLB pollution – the new task starts with cold caches.

Q. What are the states in a process life cycle?

A. New, Ready (waiting for CPU), Running (on CPU), Waiting/Blocked (waiting for I/O or an event), and Terminated. The scheduler moves processes between Ready and Running; I/O moves them to and from Waiting.

Q. User mode vs kernel mode?

A. Kernel mode has unrestricted access to hardware and all instructions; user mode is restricted and cannot directly touch hardware. Applications run in user mode and request privileged operations via *system calls*, which trap into the kernel. This protects the system from buggy or malicious programs.

Scheduling

Q. Compare common CPU scheduling algorithms.

A. **FCFS** – simple, but long jobs delay others (convoy effect). **SJF** – optimal average waiting time but needs burst prediction and can starve long jobs. **Round Robin** – fair, good for time-sharing, performance hinges on the time quantum. **Priority** – important jobs first, risks starvation (fixed by aging).

Q. What is starvation and how is it prevented?

A. Starvation is a process waiting indefinitely because others are continually preferred (e.g. low priority under priority scheduling). *Aging* prevents it by gradually raising a waiting process's priority until it runs.

Concurrency

Q. What is a race condition?

A. When two or more threads access shared data concurrently and the outcome depends on the timing/interleaving of their operations, producing incorrect results. Fixed by synchronising access to the *critical section* so only one thread modifies the shared data at a time.

Q. Difference between a mutex and a semaphore?

A. A mutex is a locking mechanism for mutual exclusion – owned by one thread, which must be the one to unlock it (binary). A semaphore is a signalling counter allowing up to N concurrent accesses; any thread may signal. Use a mutex for exclusive access, a counting semaphore to limit a resource pool.

Q. What are the conditions for deadlock and how is it handled?

A. Four must hold together: mutual exclusion, hold-and-wait, no preemption, and circular wait. Handle by *prevention* (break a condition, e.g. lock ordering), *avoidance* (Banker’s algorithm), *detection and recovery*, or *ignoring* it (the “ostrich” approach) if rare.

Q. Difference between deadlock and livelock?

A. In deadlock, processes are blocked and make no progress. In livelock, they are active and keep changing state in response to each other but still make no useful progress (like two people stepping aside in the same direction repeatedly).

Memory**Q. What is virtual memory?**

A. An abstraction giving each process its own large, contiguous address space independent of physical RAM, using paging and a disk-backed swap area. It enables running programs larger than RAM, isolation between processes, and simpler memory allocation.

Q. What is paging, and what is a page fault?

A. Paging divides memory into fixed-size pages (virtual) and frames (physical), mapped by a page table, eliminating external fragmentation. A *page fault* occurs when a referenced page is not in RAM; the OS traps, loads the page from disk (possibly evicting another), and resumes.

Q. Difference between paging and segmentation?

A. Paging splits memory into fixed-size pages (transparent to the program, no external fragmentation but some internal). Segmentation splits it into variable-size logical units (code, stack, heap) meaningful to the program (external fragmentation possible). Modern systems often combine both.

Q. What is thrashing?

A. When a system spends more time swapping pages in and out than executing, because processes collectively need more memory than is available. Detected via high paging activity; mitigated by reducing the degree of multiprogramming or adding RAM (working-set model).

4.5 Computer Networks**Models and layers****Q. Explain the OSI model layers.**

A. Seven layers, bottom to top: **Physical** (bits on the wire), **Data Link** (frames, MAC), **Network** (packets, IP, routing), **Transport** (end-to-end, TCP/UDP), **Session** (dialog control), **Presentation** (encoding/encryption), **Application** (HTTP, DNS). Mnemonic: “Please Do Not Throw Sausage Pizza Away.”

Q. Difference between the OSI and TCP/IP models?

A. OSI is a 7-layer reference model; TCP/IP is the 4-layer (Link, Internet, Transport, Application) practical model the internet actually uses. TCP/IP merges OSI’s top three into Application and the bottom two into Link. OSI is for teaching; TCP/IP is for building.

Transport

Q. TCP vs UDP?

A. TCP is connection-oriented, reliable, ordered, and flow/congestion controlled – at the cost of overhead and latency (used for web, email, file transfer). UDP is connectionless, unreliable, and unordered but fast and lightweight (used for video, VoIP, gaming, DNS). Choose by whether you need reliability or low latency.

Q. Explain the TCP three-way handshake.

A. To establish a connection: the client sends SYN, the server replies SYN-ACK, and the client sends ACK. This synchronises sequence numbers in both directions and confirms both sides can send and receive before data flows.

Q. How does TCP ensure reliability?

A. Through sequence numbers (ordering), acknowledgements and retransmission (lost-packet recovery), checksums (corruption detection), flow control via a sliding window (don't overwhelm the receiver), and congestion control (don't overwhelm the network).

Q. What is the difference between flow control and congestion control?

A. Flow control prevents a fast sender from overrunning a slow *receiver*, using the receiver's advertised window. Congestion control prevents overloading the *network* itself, using algorithms like slow start and AIMD (additive increase, multiplicative decrease).

Application and addressing

Q. What happens when you type a URL and press Enter?

A. DNS resolves the domain to an IP; the browser opens a TCP connection (and a TLS handshake for HTTPS); it sends an HTTP request; the server responds; the browser parses HTML and fetches further resources (CSS, JS, images), then renders. Caching, redirects, and keep-alive connections optimise the steps.

Q. What is DNS and how does resolution work?

A. DNS maps human-readable domain names to IP addresses. Resolution is hierarchical: the resolver queries a root server, then the TLD server (e.g. `.com`), then the domain's authoritative server, caching results along the way to speed future lookups.

Q. Difference between HTTP and HTTPS?

A. HTTPS is HTTP over TLS: it encrypts the traffic, authenticates the server via certificates, and protects integrity, preventing eavesdropping and tampering. HTTP is plaintext. HTTPS adds a TLS handshake but is now the default for the web.

Q. TCP vs UDP – which does HTTP use, and is that changing?

A. HTTP/1.1 and HTTP/2 run over TCP. HTTP/3 runs over QUIC, which is built on *UDP*, to avoid TCP's head-of-line blocking and speed up connection setup – an example of choosing UDP plus custom reliability for performance.

Devices and security

Q. Difference between a router, a switch, and a hub?

A. A hub broadcasts to all ports (Layer 1, dumb). A switch forwards frames only to the destination port using MAC addresses (Layer 2). A router connects different networks and forwards packets by IP address, choosing routes (Layer 3).

Q. What is NAT?

A. Network Address Translation maps many private IP addresses to one (or few) public IPs at the router, conserving scarce IPv4 addresses and adding a layer of isolation. The router rewrites addresses/ports and tracks the mapping to route replies back.

Q. What is the difference between symmetric and asymmetric encryption?

A. Symmetric uses one shared secret key for both encryption and decryption – fast, but key distribution is hard. Asymmetric uses a public/private key pair – solving key distribution but slower. TLS combines them: asymmetric to exchange a session key, then symmetric for bulk data.